DxPS

Real-time Provisioning System

TO-BE High Level Design (HLD) & Low Level Design (LLD)

Multi-tenant (MNO + MVNO), Kafka-driven, TM Forum-aligned, Go + PostgreSQL, 5G / 5G-Advanced ready

Item	Value
Document	DXPS-TOBE-HLD-LLD
Version	1.2 (Draft for architecture review)
Date	17 October 2026
AS-IS baseline	Legacy ITS/Huawei provisioning engine (source tree src/prv: TABS / MedProv 6.11.3 engine, SIM 6.5.6.3, PIL 1.1, adaptors, simulators)
Tenancy	Multi-tenant: host MNO and MVNOs. Every table carries tenant varchar(40); primary key is (tenant, id uuid)
Target stack	Go (latest stable), Apache Kafka 4.x (KRaft), PostgreSQL 18+, Kubernetes
Standards	TM Forum ODA + Open APIs (TMF641/652/702/640/638/639/633/634/688/701/630), 3GPP Rel-18/19 SBA, IETF NETCONF/RESTCONF/YANG, OpenConfig gNMI, O-RAN O1, BBF TR-369/TR-385, GSMA SGP.22/SGP.32, GSMA Open Gateway (CAMARA)
Status	Draft - for review

Table of Contents

1. Executive Summary 4
2. Scope, Goals and Requirements 4
2.1 In scope 4
2.2 Out of scope 4
2.3 Functional requirements 4
2.4 Non-functional requirements 5
3. AS-IS Analysis (legacy ITS/Huawei engine) 6
3.1 Component inventory 6
3.2 AS-IS data model 7
3.3 Ordering, priority and transaction handling (AS-IS) 7
3.4 AS-IS status code vocabulary 7
3.5 AS-IS protocols and target replacement 7
3.6 Pain points and how DxPS addresses them 8
4. Architecture Principles and Standards 9
4.1 Principles 9
4.2 TM Forum alignment 9
5. High Level Design 10
5.1 Logical architecture 10
5.2 Multi-tenancy (host MNO and MVNOs) 11
5.2.1 Tenant identity and propagation 11
5.2.2 Fairness and noisy-neighbour protection 11
5.2.3 Tenant-specific behaviour 11
5.3 Removing PIL 12
5.4 Kafka design 12
5.5 Command ordering 13
5.6 Priorities 13
5.7 Transaction management 14
5.8 Many-to-many relations 15
5.9 Low-latency strategy 15
5.10 External network coverage (modern protocols only) 16
5.11 Deployment architecture 16
5.12 Security 16
5.13 Observability and operations 17

5.14 Operations and configuration console 17
6. Low Level Design 19
6.1 Go code base layout 19
6.2 Message contracts (protobuf) 19
6.3 Kafka client and broker configuration 20
6.3.1 Producer (franz-go equivalents) 20
6.3.2 Consumer 20
6.3.3 Broker / topic 21
6.4 Tiered consumer and weighted-fair scheduler (Go) 21
6.5 Key-lane executor (ordering with parallelism) 22
6.6 Orchestrator: BC ingestion step 23
6.7 Entity sequencer 24
6.8 Result handling, retries, compensation 24
6.9 Adapter SPI 25
6.10 Catalog / decomposition (replaces the legacy MMLGEN templates) 26
6.11 PostgreSQL physical design (multi-tenant) 27
6.12 Northbound API examples 30
6.13 Error model 31
6.14 Performance and capacity model 32
6.15 Testing strategy 32
6.16 Console API and web UI 32
7. Migration Strategy (legacy engine -> DxPS) 33
8. Requirements Traceability 33
9. Risks and Open Decisions 33
Appendix A - Glossary 34
Appendix B - References 34

1. Executive Summary

DxPS (Real-time Provisioning System) is the target platform that replaces the legacy ITS/Huawei provisioning engine. It is multi-tenant from the ground up, so one deployment serves the host MNO and any number of MVNOs with isolated data, quotas, catalogs and notifications.

The legacy engine is database-centric. A CRM-facing dispatcher (PIL) copies business commands into an Oracle interface table. The MMLGEN daemon expands them into vendor MML through templates, and a fleet of single-threaded SIM/HLRCOMM processes polls Oracle and drives network elements over TCP/Telnet, X.25, async serial, FIFOs, gSOAP, CORBA and SMPP. Throughput is set by poll intervals and row locks. Ordering and priority depend on SQL (min(COMMAND_PRIORITY), DEPENDENCY_SEQ, status 9/10). Business-to-network mapping is effectively 1:N.

DxPS replaces this with an event-driven, horizontally scalable platform:

- **Multi-tenant for MVNOs.** Every table carries a tenant column (varchar 40), and the primary key is (tenant, id uuid). The tenant comes from the caller's OAuth2 token and travels in every Kafka message. PostgreSQL row-level security enforces isolation, and quotas are applied per tenant and per shared network element.
- **Kafka-first ingestion.** Business Commands are published straight to Kafka priority-tier topics, by the CRM/COM or through a TMF641/TMF652 API gateway. PIL is removed.
- **DxPS Orchestrator (Go).** Per-tenant, per-entity sequencing, catalog-driven decomposition into a task DAG, saga transaction management with compensation, and full many-to-many Business Command <-> Network Task relations (including coalescing).
- **PostgreSQL.** System of record for orders, plans, tasks, catalog and idempotency. A transactional outbox with fast-publish gives atomic state change plus event emission at low latency.
- **Protocol-specific Go adapters.** Modern network interfaces only: 3GPP 5G SBA (HTTP/2 + JSON, OAuth2), IMS/VoNR provisioning REST, NETCONF/RESTCONF/YANG, gNMI, 3GPP TS 28.532 MnS, O-RAN O1, BBF TR-369 USP and TR-385, GSMA SGP.22/SGP.32 eSIM (ES2+), CAMARA/NEF exposure and TMF Open APIs towards BSS.
- **Latency and scale.** Platform overhead target is p99 <= 25 ms from Kafka ingest to NE request. Throughput scales linearly with partitions and pods (baseline 5,000 BC/s, 25,000 NE tasks/s per cluster).

2. Scope, Goals and Requirements

2.1 In scope

- AS-IS study of the legacy ITS/Huawei engine (source tree src/prv): engine (MMLGEN, SIM), dispatchers (PIL), adaptors (BlackBerry, CORBA, NPM, SMSC), simulators.
- DxPS HLD: architecture, multi-tenancy (MNO + MVNO), Kafka topology, ordering, priority, transaction model, data model, deployment, security, observability.
- DxPS LLD: Go service design, Kafka client/broker configuration, PostgreSQL DDL, algorithms, state machines, adapter SPI, APIs, error model, testing and migration.

2.2 Out of scope

- CRM/COM product ordering (TMF622) redesign.
- Charging/rating engine redesign.
- Vendor-specific NE configuration beyond adapter contracts.

2.3 Functional requirements

ID	Requirement
FR-01	Business Commands (BC) are received from Kafka. A TMF641/652 REST/gRPC facade publishes to the same topics.

FR-02	No PIL: no CRM-DB polling or intermediate interface table.
FR-03	Strict per-entity command order (subscriber / service / resource), with a configurable override for preemptive commands such as barring.
FR-04	At least four priority classes, with starvation protection.
FR-05	Transaction management across multiple NEs: all-or-nothing (saga + compensation), partial-allowed, or best-effort, chosen per command spec.
FR-06	Many-to-many relations: Order <-> BC <-> Network Task <-> NE <-> Endpoint, and Service <-> Resource.
FR-07	Catalog-driven decomposition (versioned, per tenant with GLOBAL defaults) replacing the legacy MMLGEN templates.
FR-08	Modern Telco protocol adapters covering 5G SA/5G-Advanced, IMS/VoNR, transport/IP, RAN, FTTH, eSIM, BSS.
FR-09	TMF688 event notifications for order/item state changes, plus inventory synchronisation (TMF638/639).
FR-10	Idempotent re-submission (Idempotency-Key) and exactly-once effect at the NE where the NE allows it.
FR-11	Cancel / amend in-flight orders (TMF641 cancelServiceOrder).
FR-12	Multi-tenancy for MVNOs: every table has tenant varchar(40) NOT NULL; primary key (tenant, id uuid); every API, Kafka message, log line and metric is tenant-scoped.
FR-13	Tenant onboarding without code change: tenant record, catalog overrides, NE entitlements, quotas, OAuth2 client and TMF688 hub.
FR-14	Shared and dedicated network elements: a light MVNO uses host-MNO NEs under quota; a full MVNO can register its own NEs.

2.4 Non-functional requirements

ID	Category	Target
NFR-01	Latency (platform)	p50 <= 8 ms, p99 <= 25 ms from BC append to first NE request (excludes NE time)
NFR-02	Latency (P0 end-to-end)	p99 <= 150 ms for single-NE P0 commands such as barring on UDM (NE RTT <= 50 ms)
NFR-03	Throughput	>= 5,000 BC/s and >= 25,000 NE tasks/s per cluster; linear scale-out
NFR-04	Availability	99.99% platform; zero data loss (RPO 0 in-region, RPO <= 5 s cross-region)
NFR-05	Durability	Kafka RF=3, min.insync.replicas=2, acks=all; PostgreSQL synchronous standby
NFR-06	Ordering	No reordering for the same (tenant, entity_key) under failover, rebalance or retry
NFR-10	Tenant isolation	No cross-tenant read/write possible (RLS + composite keys + token-derived tenant); noisy-neighbour protection so one tenant's bulk load cannot breach another tenant's P0/P1 SLO
NFR-07	Security	mTLS everywhere, OAuth2 (3GPP TS 33.501 / NRF tokens), secrets in Vault/KMS, audit trail
NFR-08	Operability	OpenTelemetry traces end-to-end (traceparent in Kafka headers), SLO dashboards, runbooks
NFR-09	Elasticity	Autoscale on consumer lag (KEDA) and CPU; no stateful pods except Kafka/PostgreSQL

3. AS-IS Analysis (legacy ITS/Huawei engine)

This chapter describes the legacy system only. Its table and module names (PRV_*, PIL, MMLGEN, SIM) are quoted as they appear in the legacy source tree (src/prv) and are not used by DxPS.

Product lineage: TABS / MedProv (International Turnkey Systems). Rollback feature MedProv 6.11.3, SIM 6.5.6.3, PIL 1.1. Languages: C/C++ (Pro*C), Java (SMSC, CORBA client), gSOAP 2.7, IONA Orbix 6.2. Targets: Solaris 10 and HP-UX Itanium. Source control: Visual SourceSafe. The unit_test folder is empty.

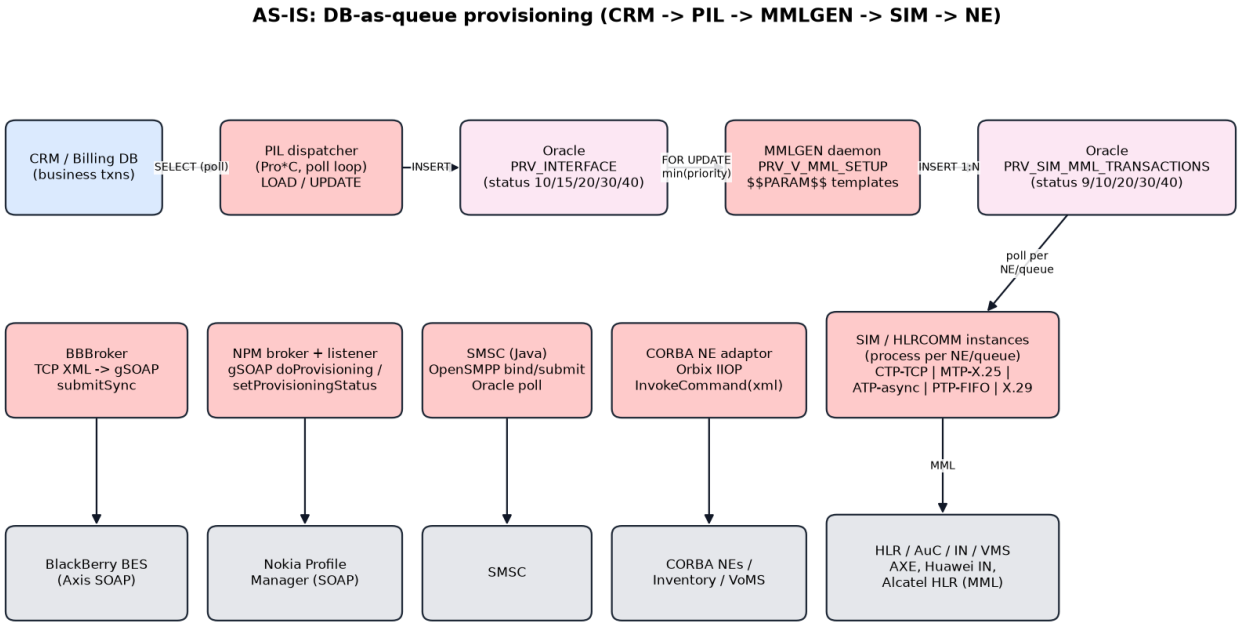


Figure 1 - AS-IS end-to-end flow

3.1 Component inventory

Component	Location	Technology	Role
PIL	dispatchers/PIL (PILMain.pc, CCRMInterface.pc, CPRVInterface.pc, CXMLBCParser, CMMLBCParser)	Pro*C, single thread, sleep loop	Polls CRM via CRM_FETCH_QUERY and inserts PRV_INTERFACE (LOAD); writes status back to CRM (UPDATE); DUAL mode
MMLGEN	engine/MMLGEN (MMLGen.pc, CPRVInterface.pc, CTranslator.pc, CTranslationSetup.pc, CQueue)	Pro*C/C++ daemon	Claims a pending transaction (FOR UPDATE, min COMMAND_PRIORITY); expands BC via templates into PRV_SIM_MML_TRANSACTIONS
SIM / HLRCOMM	engine/SIM (hlrmain.c, hlrspc.pc, hlrtcp.c, hlrmtp.c, hlrx25.c, hlrasyn.c, hlrpip.c)	C, one process per NE/queue	Polls NE command rows, runs a login/send/parse session per NE, updates status, promotes the next command
BBBroker	adaptors/blackberry	C++ TCP server + gSOAP client	BlackBerry BES submitSync (SOAP RPC/encoded)
NPM broker / listener	adaptors/npm (INPMREQUESTBROKER, INPMRESPONSELISTENER)	C++ gSOAP client + server	Nokia Profile Manager doProvisioning / getSubscriptions, async setProvisioningStatus callback
SMSC adaptor	adaptors/SMSC/src/jhlrcsms	Java, OpenSMPP, Oracle pool	Polls Oracle and submits SMS via SMPP bind-transmitter
CORBA NE adaptor	adaptors/CORBA	Orbix IIOP, CosNaming, XA	Generic INEAdaptor::InvokeCommand(xml); Inventory, SmartComm, VoMS
Simulators	simulators/Simulator, IN_SIM, NPM	C TCP servers, gSOAP	HLR MML simulator, CMD->RESP stub, NPM async simulator

3.2 AS-IS data model

Table / View	Purpose
PRV_INTERFACE	Business command queue: TRANSACTION_ID, INTERFACE_ID, BUS_CMD_ID, COMMAND_ORDER(+ARGUMENT), COMMAND_PRIORITY, MSISDN, AREA, mapped params, TRANSACTION_STATUS, RETURN_CODE, RESPONSE_MESSAGE, ARCHIVE_FLAG
PRV_SIM_MML_TRANSACTIONS	NE command queue: COMMAND_ID (SIMMMLTRNS_SEQ), TRANSACTION_ID, NE_ID, PROTOCOL_ID, BUS_CMD_ID, SIMCMD_QUEUE, SIMCMD_STATUS, COMMAND_TEXT, IS_ROLLBACK, COMMAND_NR, MMLCMDSET_ID
PRV_V_MML_SETUP	Templates keyed by COMMAND_ORDER, ARGUMENT and MSISDN range; ORDER BY DEPENDENCY_SEQ, COMMAND_NR
PRV_V_NEXT_QUEUE	NE/protocol -> SIM queue routing
PRV_SETUP	Key/value configuration including dynamic SQL (CRM_FETCH_QUERY, CRM_UPDATE_QUERY), BC_PRV_PARAM_MAP
PRV_SIM_CONFIG / _INSTANCES / _CONFIG_PARAMS	Per-SIM process binding to NE + protocol + queue, poll and delay knobs
PRV_NETWORK_ELEMENTS, PRV_PROTOCOLS(_PARAMS/_ADDRESSES)	NE and protocol registry, login endpoints
PRV_RESPONSE_MESSAGES, PRV_CUSTOM_MESSAGES	NE response string -> success/fail/custom code mapping
PRV_VW_ROLLBACK_NECMDS, PRV_VW_TRANS_EXECUTED_NECMDS	Rollback generation and validation (6.11.3)

3.3 Ordering, priority and transaction handling (AS-IS)

Concern	AS-IS mechanism	Weakness
Priority	MMLGEN FetchNext: COMMAND_PRIORITY = (SELECT min(COMMAND_PRIORITY) ...) then min INTERFACE_ID, FOR UPDATE	Index-hinted full scans; strict priority causes starvation; one claimer at a time
Ordering	First NE cmd status 10 (READY), others 9 (NOT_READY); SIM promotes next min COMMAND_ID after success	Sequential per transaction, even for independent NEs; no cross-transaction per-subscriber guarantee
Dependency	DEPENDENCY_SEQ, COMMAND_NR in template view	Linear only; no DAG and no parallel branches
Transactions	oracommit() at status boundaries; rollback mode generates compensating cmds (49/50/55/95 -> 70/80/85)	Separate process mode (-r/-d); coarse commits; ad-hoc dual mode
Retries	Transport error -> reset to READY; file helpers retry 6/20 times	No backoff, no DLQ, possible hot loops
Relations	One TRANSACTION -> N BUS_CMD -> N NE cmds	No N:M (no coalescing, no shared tasks, no service/resource inventory)

3.4 AS-IS status code vocabulary

Code	Constant (MMLGEN.h / hlrspc.h)	Meaning	DxPS state
9	CMD_PENDING_NOT_READY	NE cmd waiting for predecessor	PLANNED
10	TRANS_PENDING / CMD_PENDING_READY	Pending / ready	acknowledged / READY
15	TRANS_BEING_PARSED	MMLGEN translating	inProgress (planning)
20	TRANS_BEING_PROCESSED / CMD_BEING_PROCESSED	In NE execution	IN_FLIGHT
30	SUCCESS	OK	SUCCEEDED / completed
40	FAIL	Failed	FAILED / failed
49 / 50	ROLLBACK_NOT_READY / ROLLBACK_READY	Rollback queued	COMPENSATING
95 / 55	ROLLBACK_GENERATING / ROLLBACK_GENERATED	Rollback cmds being built	COMPENSATING
70	ROLLBACK_SUCCEEDED	Rolled back	COMPENSATED
80 / 85	ROLLBACK_FAILED / ROLLBACK_FAILED_NONROLLBACK	Rollback failed / not rollbackable	COMPENSATION_FAILED

3.5 AS-IS protocols and target replacement

AS-IS protocol	Where	Status	DxPS replacement
MML over TCP/Telnet (CTP)	SIM hlrctp.c / hlrtcp.c	Legacy	3GPP SBA HTTP/2 (Nudr/Nudm), vendor Provisioning Gateway REST, NETCONF/YANG
MTP over X.25, X.29	hlrmtp.c, hlrx25.c, hlrx29.c	Obsolete	Retired
Async serial (ATP)	hlrasyn.c	Obsolete	Retired
FIFO / rcp file drop (PTP)	hlrpip.c, intservice.c,	Obsolete /	Retired; event or REST APIs

	vmservice.c	insecure	
SOAP RPC/encoded (gSOAP 2.7, Axis 1.x)	BlackBerry, NPM	Legacy	REST/JSON (OpenAPI 3.1), async callbacks / Kafka events
CORBA IIOP (Orbix)	adaptors/CORBA	Obsolete	gRPC (internal) / TMF Open APIs (external)
SMPP 3.4 bind-transmitter	adaptors/SMSC	Legacy	3GPP SMSF (Nsmsf) / SMS data in UDM; notifications via CPaaS REST or GSMA RCS
Oracle polling (Pro*C)	PIL, MMLGEN, SIM, SMSC	Anti-pattern	Kafka topics + PostgreSQL outbox

3.6 Pain points and how DxPS addresses them

#	AS-IS pain point	DxPS response
1	DB used as message bus (polling, FOR UPDATE contention, sleep latency)	Kafka push consumption with fetch.max.wait.ms <= 5-10 ms; PostgreSQL holds state only
2	Single-threaded process per NE/queue	Go goroutine worker pools with key-lane parallelism; HTTP/2 multiplexing
3	Strict priority with starvation	Priority-tier topics + weighted-fair scheduler with aging
4	Linear dependency only	DAG plan with parallel branches and explicit edges
5	Rollback as a separate process mode	Built-in saga with per-task compensation spec
6	Stringly-typed templates ($$PARAM$$, {#FUN#})	Versioned catalog with JSON Schema-validated params and CEL expressions
7	Obsolete transports and SOAP/CORBA	Modern protocol adapters only (see section 5.10)
8	No end-to-end tracing	OpenTelemetry trace context propagated in Kafka headers and NE calls
9	Solaris/HP-UX lock-in, VSS	Containers on Kubernetes, Git, CI/CD, IaC
10	PIL tightly coupled to CRM schema	PIL removed; CRM produces events or calls TMF641
11	Single-operator design: no tenant concept, one CRM, one catalog	Native multi-tenancy: tenant in every table and message, per-tenant catalogs, quotas, hubs and RLS

4. Architecture Principles and Standards

4.1 Principles

- **Tenant everywhere.** tenant is a mandatory first-class attribute of every API call, Kafka message, table row, cache key, log line, metric and trace. Nothing is processed without it.
- **Event-first, state-in-DB.** Kafka carries intent and facts; PostgreSQL is the system of record for order/plan/task state.
- **Ordering by key, priority by topic.** Kafka guarantees order per partition. The tenant|entity_key decides the partition; the priority tier decides the topic.
- **Exactly-once effect.** Idempotent producers, Kafka transactions for Kafka-to-Kafka steps, inbox dedupe, PostgreSQL outbox, NE idempotency keys.
- **Catalog-driven.** No code change to add a product or command. Decomposition rules live in a versioned catalog (TMF633/634).
- **Adapters behind an SPI.** One Go interface for every network domain; protocol detail stays inside the adapter.
- **No legacy protocols.** X.25, async, FIFO, CORBA, SOAP-encoded and Telnet MML are not supported in the target. Any surviving legacy NE is reached through a vendor modern northbound API or retired.
- **Cloud-native.** Stateless Go services on Kubernetes, 12-factor configuration, GitOps.

4.2 TM Forum alignment

DxPS implements the ODA functional blocks Service Order Management, Resource Order Management, Resource Activation & Configuration and Service/Resource Inventory. It follows the TMF630 REST API Design Guidelines and the SID (Information Framework) entities.

TMF asset	Use in DxPS
TMF641 Service Ordering	Northbound order API (ServiceOrder / ServiceOrderItem), states acknowledged -> inProgress -> completed/failed/partial/cancelled
TMF652 Resource Ordering	ResourceOrder for resource-level commands (SIM/eSIM, numbers, ports)
TMF640 Service Activation & Configuration	Synchronous service activation for P0/P1 single-step commands
TMF702 Resource Activation & Configuration	Resource-level activation semantics for adapters (internal contract mirrors it)
TMF633 / TMF634 Service / Resource Catalog	Command specs, decomposition rules, compensation rules per tenant with GLOBAL defaults (replaces the legacy MML setup templates)
TMF638 / TMF639 Service / Resource Inventory	Post-activation inventory sync; source for amend/cease delta calculation
TMF688 Event Management	Hub/listener notifications (ServiceOrderStateChangeEvent, ResourceOrderStateChangeEvent, ...)
TMF701 Process Flow	Exposes orchestration plan and task flow for monitoring
TMF642 Alarm Management	Raises alarms for compensation failures, NE circuit-open, DLQ growth
TMF654 / TMF666	Prepay balance and account management towards OCS/BSS adapters
TMF645 Service Qualification	Optional pre-check (feasibility) before planning
TMF921 Intent Management	Future: intent-based orders (for example slice SLAs)
SID	Entity naming: ServiceOrder, ServiceOrderItem, ResourceOrder, Service, Resource, ServiceSpecification, ResourceSpecification, PartyRole (MVNO tenant = Party with role 'MVNO')
TMF632 / TMF669 Party / Party Role	Tenant (MVNO) master data reference for onboarding
eTOM	Service Configuration & Activation, Resource Provisioning process areas

5. High Level Design

5.1 Logical architecture

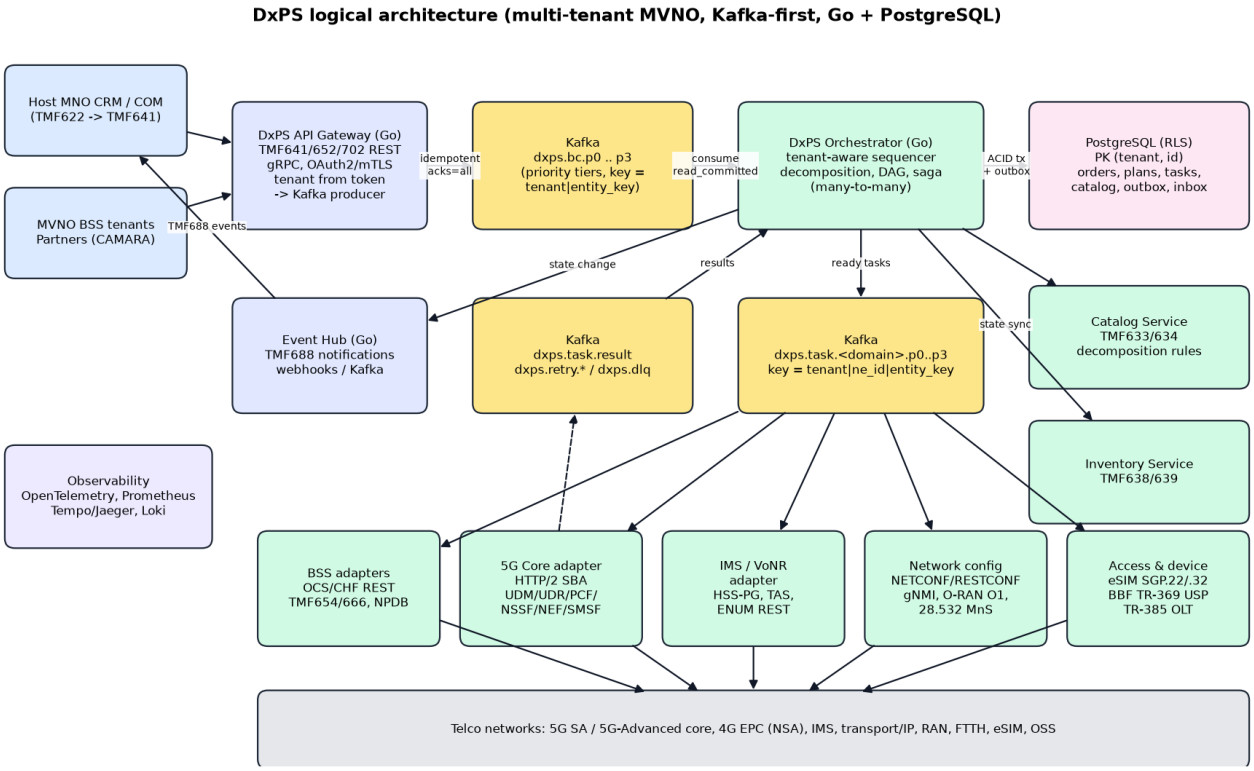


Figure 2 - DxPS logical architecture

Component	Responsibility	Scaling unit
DxPS API Gateway (Go)	TMF641/652/640 REST (OpenAPI 3.1) and gRPC; authN/authZ; tenant resolved from the OAuth2 token; per-tenant rate limits; schema validation; Idempotency-Key; produce BC to Kafka; return acknowledged	Stateless pods (HPA on RPS)
Business Command topics	dxps.bc.p0..p3; key = tenant|entity_key; protobuf with Schema Registry	Partitions
DxPS Orchestrator (Go)	Consume BC; tenant check; inbox dedupe; entity sequencing; decomposition via tenant catalog; DAG plan; saga control; consume results; emit next tasks; compensation; order state	Pods = consumer group members (KEDA on lag)
DxPS Catalog Service (Go)	Versioned command specs per tenant (GLOBAL defaults + tenant overrides), decomposition rules, parameter schemas, compensation specs; hot-reloaded into orchestrator memory	Read-mostly; cached
DxPS Tenant Service (Go)	Tenant registry, onboarding, NE entitlements (tenant_ne_access), quotas, SLA tier, OAuth2 client mapping	Read-mostly; cached
Task topics	dxps.task.<domain>.p0..p3; key = tenant|ne_id|entity_key	Partitions per domain
DxPS Adapter workers (Go)	Domain SPI implementations; per-NE and per-tenant rate limits; circuit breaker; NE session/HTTP2 pools; response normalisation	Pods per domain (KEDA on lag)
Result topic	dxps.task.result; key = tenant|order_id	Partitions
DxPS Event Hub (Go)	TMF688 hub registration per tenant; webhook delivery with retries only to that tenant's listeners; Kafka event topic for internal consumers	Stateless pods
DxPS Inventory Service	TMF638/639; per-tenant projections of activated services/resources	PostgreSQL schema
Outbox Relay	Safety net that publishes unpublished outbox rows (fast-publish handles the normal path)	1-3 pods, leader-elected per shard
DxPS Console (Go API + Next.js UI)	Signed-in operations and configuration console for all tenants: orders, DLQ redrive, NE faults, catalog drafts and Template Lab, tenant and NE inventory edits, reports, console users with roles (section 5.14)	1 API + 1 UI instance; stateless UI
PostgreSQL	System of record (tenants, orders, BC, plans, tasks, links, catalog, outbox/inbox, audit); PK (tenant, id) on every table; row-level security	Primary + sync standby + read replicas; partitioned by tenant

5.2 Multi-tenancy (host MNO and MVNOs)

DxPS serves the host MNO and its MVNOs from one deployment. A tenant is any operator brand whose orders, subscribers and configuration must be isolated from the others.

Tenant type	Description	Network elements	Typical isolation
HOST	Host MNO that owns the radio and core network	Owns all shared NEs	Shared pool, highest default quota
FULL_MVNO	MVNO with its own core elements (own UDM/HSS, PCF, OCS) on host RAN	Own NEs registered under its tenant, plus shared host NEs it is entitled to	Shared pool or dedicated PostgreSQL partition / Kafka topics
LIGHT_MVNO	Branded reseller / service provider using host core	Host NEs only, through tenant_ne_access with quota	Shared pool
ENTERPRISE / IOT	Private network or IoT customer slice	Host NEs + dedicated slice (S-NSSAI) or private core	Shared pool, slice-scoped

5.2.1 Tenant identity and propagation

- **Identifier:** tenant varchar(40), pattern ^[A-Za-z0-9_.-]{1,40}$, immutable (for example 'host-mno', 'mvno-alpha'). The reserved value 'GLOBAL' holds shared catalog defaults only.
- **API:** the gateway takes tenant from the OAuth2 access token claim (tenant), never from the request body. Host-operator admin clients with scope dxps:tenant:any may act for a tenant through the X-Tenant header, and every such call is audited.
- **Kafka:** tenant is a field in every message and a record header, and the first part of every key (tenant|entity_key). MVNO systems publish through the gateway. Native Kafka producers are allowed for the host MNO, and for large MVNOs on dedicated topics (dxps.t.<tenant>.bc.<tier>) protected by prefixed ACLs.
- **Database:** every table has tenant varchar(40) NOT NULL and PRIMARY KEY (tenant, id uuid). All foreign keys are composite (tenant, x_id), so a row can never reference another tenant's row.
- **Runtime:** every transaction starts with SET LOCAL app.tenant, and row-level security policies restrict reads and writes to that tenant (section 6.11).
- **Observability:** tenant is a trace attribute, a structured log field and a metric label (bounded cardinality: one label value per tenant).

5.2.2 Fairness and noisy-neighbour protection

- Gateway: per-tenant request quotas (token bucket per tenant and tier) from the tenant SLA.
- Orchestrator: inside each priority tier the scheduler serves tenants by deficit round robin, weighted by SLA, so one MVNO's bulk migration cannot starve another tenant.
- Shared NEs: each NE has a global token bucket (max_tps) plus a per-tenant bucket from tenant_ne_access.quota_tps. P0/P1 capacity is reserved per tenant.
- Kafka: client quotas (produce/fetch byte rate) per tenant principal for native producers.
- PostgreSQL: very large tenants get their own LIST partitions, so their vacuum and index load does not affect others.

5.2.3 Tenant-specific behaviour

- Catalog resolution: tenant spec version first, then GLOBAL. An MVNO can override decomposition (for example its own OCS) without affecting others.
- NE selection: neSelect() only returns NEs the tenant owns or is entitled to (tenant_ne_access), filtered by operation.
- Subscriber data on shared NEs is tagged with the tenant (for example a UDR/PCF subscriber group or a dedicated S-NSSAI / DNN), so the host network can apply per-MVNO policy.
- Notifications: TMF688 hubs are registered per tenant, and events are only delivered to that tenant's listeners.

- Onboarding: create the tenant, OAuth2 client, quotas, NE entitlements, catalog overrides and hub, all through APIs with no deployment.

5.3 Removing PIL

PIL existed to copy rows from CRM tables into the legacy interface table and copy status back. In DxPS:

- The host CRM/COM publishes BusinessCommand events directly to dxps.bc.<tier> (native producer). MVNO BSS systems, and any system that cannot run a Kafka client, call the TMF641 API gateway, which publishes on their behalf with the tenant from their token.
- Status flows back as TMF688 events (dxps.event.tmf688 and webhooks), not by UPDATE on CRM tables.
- Parameter mapping (the legacy BC parameter map) moves into the catalog as a JSON-Schema-validated parameter contract per command spec and tenant.
- Validation that PIL did (unique MSISDN/area per transaction) moves to the gateway (syntax) and the orchestrator (semantic).

5.4 Kafka design

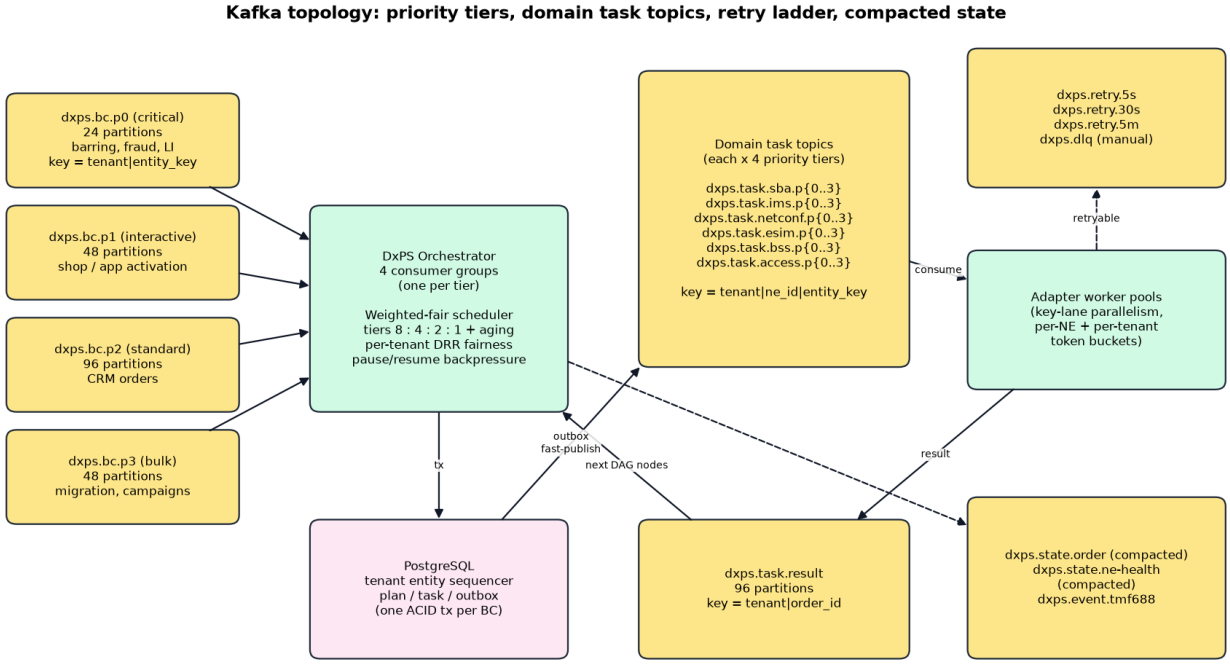


Figure 3 - Kafka topology

Topic	Partitions	Key	Retention / cleanup	Notes
dxps.bc.p0	24	tenant|entity_key	7d delete	Critical: barring, fraud lock, lawful/regulatory, emergency
dxps.bc.p1	48	tenant|entity_key	7d delete	Interactive: retail/app activation, SIM swap
dxps.bc.p2	96	tenant|entity_key	7d delete	Standard CRM / MVNO BSS orders
dxps.bc.p3	48	tenant|entity_key	3d delete	Bulk: migrations, MVNO onboarding loads, campaigns
dxps.t.<tenant>.bc.p0..p3	12-48	tenant|entity_key	7d delete	Optional dedicated ingress for large MVNOs (prefixed ACLs, own quotas)
dxps.task.<domain>.p0..p3	24-96	tenant|ne_id|entity_key	3d delete	domain = sba, ims, netconf, esim, bss, access, exposure
dxps.task.result	96	tenant|order_id	3d delete	Adapter -> orchestrator results
dxps.retry.5s / 30s / 5m	24 each	tenant|ne_id|entity_key	1d delete	Delayed retry ladder (not-before

				header)
dxps.dlq	12	original key	30d delete	Manual inspection and replay tooling (tenant-filtered)
dxps.state.order	48	tenant|order_id	compact	Latest order snapshot for read models and DR
dxps.state.ne-health	12	tenant|ne_id	compact	Circuit state, capacity and health shared across adapters
dxps.state.tenant	6	tenant	compact	Tenant config, quotas and entitlements broadcast to all pods
dxps.event.tmf688	48	tenant|order_id	7d delete	TMF688 events, delivered per tenant hub

Partition counts are sized for 3x headroom over NFR-03. They follow partitions >= target msg/s / per-partition consumer rate, with a per-partition rate of about 1-2k BC/s for the orchestrator. Partitions can only be increased, and doing so remaps keys. Increase only during a drained maintenance window, or create the topic with the final count up front (recommended).

5.5 Command ordering

- **Producer side.** Every BC carries tenant, entity_key (canonical subscriber/service key such as SUPI/IMSI, MSISDN, service id or account id) and a monotonic entity_seq from the source system. The record key is tenant|entity_key, and the default Kafka partitioner (murmur2 on key) sends all of an entity's commands to one partition of a tier. The same MSISDN under two tenants (for example after porting between MVNOs) is two different sequences.
- **Idempotent producer.** enable.idempotence=true with max.in.flight.requests.per.connection <= 5 keeps order within a partition under retries.
- **Cross-tier ordering.** The same entity can receive commands on different tiers, so Kafka order alone is not enough. The orchestrator's entity sequencer (PostgreSQL table entity_sequencer, unique on (tenant, entity_key)) admits BC n only after BC n-1 is terminal. Early arrivals are parked (state=pending) and released in sequence.
- **Preemption.** A command spec can declare preempt=true (for example BAR_ALL). It overtakes queued commands for that entity, and parked commands are re-validated against the new state (cancel/supersede rules).
- **Multi-entity orders.** Commands touching several entities (for example a family plan) take sequencer slots for every entity in canonical sort order, which avoids deadlock.
- **Per-NE order.** Task key tenant|ne_id|entity_key keeps tasks for one entity on one NE in order. DAG edges impose order across NEs.
- **Rebalance safety.** Cooperative incremental rebalancing (KIP-848 consumer protocol in Kafka 4.x) plus static membership. Offsets are committed only for contiguously completed records.

5.6 Priorities

Kafka has no message priority, so priority is implemented by tier topics plus a scheduler:

Tier	Examples	Weight	Latency SLO (platform p99)	Producer linger
P0 Critical	Barring/unbarring, fraud lock, LI, emergency service, SIM swap security lock	8 (+ strict pre-emption)	10 ms	0 ms
P1 Interactive	Shop/app activation, eSIM download, plan change by customer	4	25 ms	1 ms
P2 Standard	CRM orders, B2B orders, scheduled changes	2	100 ms	5 ms
P3 Bulk	Migration, mass re-provisioning, campaigns	1 (throttled)	best effort	20 ms

- Each tier has its own consumer group, so lag and autoscaling are independent. Records flow into one in-process weighted-fair scheduler with per-tier bounded queues.

- Aging: a P3 item waiting longer than its max_wait (default 60 s) is promoted one tier for scheduling. This prevents starvation.
- Backpressure: when a tier queue is full, the consumer calls PauseFetchTopics and resumes below a low-water mark. No unbounded memory.
- Within a tier, tenants are served by weighted deficit round robin (weight from the tenant SLA tier), so tier priority and tenant fairness are both enforced.
- Adapters apply the same tiering on task topics. A per-NE token bucket reserves capacity (for example 20%) for P0/P1, and per-tenant buckets on shared NEs stop one tenant from saturating an NE.
- Priority never breaks per-entity order, except where preempt=true (section 5.5).

5.7 Transaction management

There are three layers of consistency:

Layer	Mechanism	Guarantee
Local (PostgreSQL)	One ACID transaction per orchestration step: inbox insert + state change + task rows + outbox rows	Atomic state + intent to publish
Kafka <-> PostgreSQL	Inbox dedupe (per-service inbox, PK (tenant, id = message_id)) and transactional outbox with fast-publish + relay	Effectively-once processing; no lost or duplicated events
Kafka -> Kafka (adapters)	Kafka transactions: consume task, produce result, commit offset atomically (transactional.id per partition owner, isolation.level=read_committed)	Exactly-once result emission
Cross-NE (business)	Orchestrated saga over the task DAG; per-task compensation spec; pivot task concept	All-or-nothing, partial, or best-effort per command spec
NE effect	Idempotency key per task (HTTP header / NETCONF confirmed-commit / PUT semantics); read-before-write when the NE lacks idempotency	Exactly-once effect where supported; safe retry elsewhere

- **Transaction modes (per command spec):** ATOMIC (full compensation on any failure), PARTIAL_ALLOWED (TMF641 state partial, no compensation of succeeded items), BEST_EFFORT (retry only).
- **Pivot task:** Tasks after the pivot (for example SM-DP+ ConfirmOrder or customer notification) are not compensated. Failures after the pivot are retried forward.
- **Compensation order:** Reverse topological order of succeeded tasks. Compensation tasks use P0/P1 tiers to shorten the inconsistency window.
- **NETCONF:** Candidate datastore + confirmed-commit with confirm-timeout gives native per-NE atomicity; the saga coordinates across NEs.

5.8 Many-to-many relations

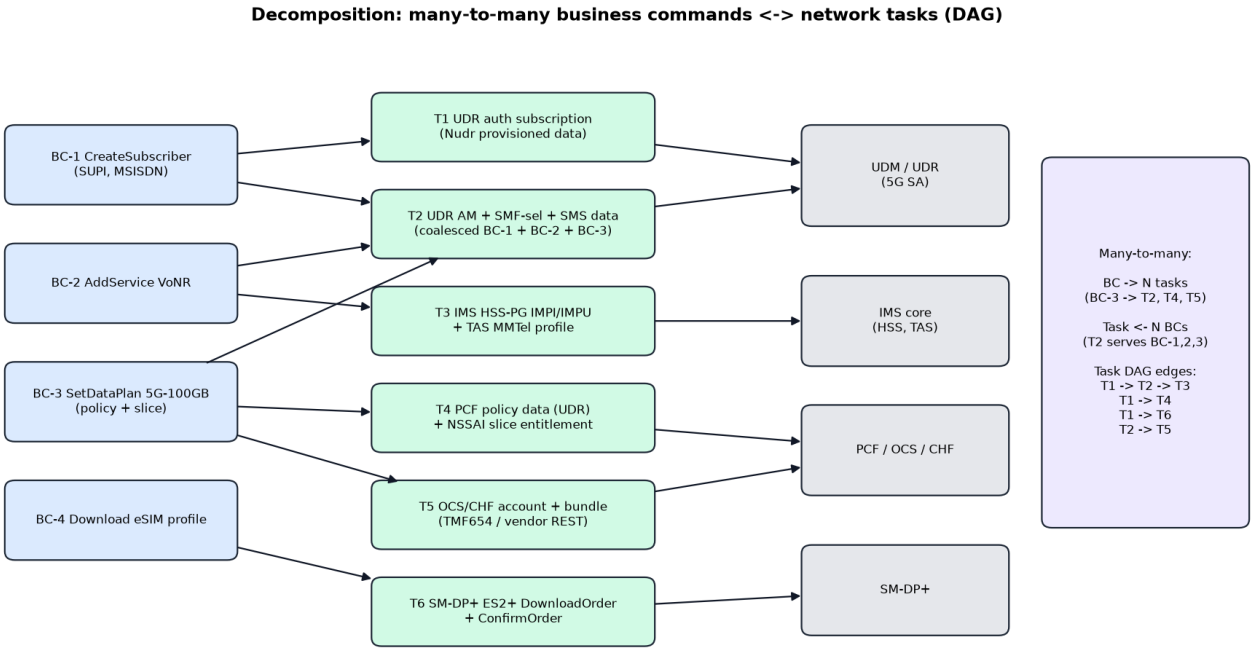


Figure 4 - Many-to-many decomposition into a task DAG

- **Tenant <-> everything:** 1:N. Every row belongs to exactly one tenant, and all relations are composite (tenant, id), so relations never cross tenants.
- **Tenant <-> NE:** N:M via tenant_ne_access. A shared host NE serves many tenants, each under its own quota and allowed operations; a full MVNO also owns its own NEs.
- **Order <-> BC:** 1:N. One order holds many BCs; a BC belongs to one order.
- **BC <-> Task:** N:M via bc_task_link. One BC produces many tasks, and one task can satisfy many BCs (coalescing: several BCs that change the same UDR resource merge into one PATCH).
- **Task <-> Task:** DAG via task_dependency (hard = must succeed, soft = must finish).
- **Entity <-> BC:** 1:N with ordering through entity_sequencer. Multi-entity BCs link through bc_entity.
- **NE <-> Endpoint <-> Adapter:** N:M. One NE exposes several interfaces (for example UDM SBA + vendor PG); one adapter type serves many NEs; endpoints carry weight and health for load balancing and geo-redundancy.
- **Service <-> Resource:** N:M in inventory (TMF638/639 relationships), used for amend/cease delta calculation.

5.9 Low-latency strategy

Stage	Budget p99	Technique
Gateway -> Kafka append	3 ms	acks=all with in-AZ ISR, linger 0-1 ms for P0/P1, lz4, keep-alive connections
Kafka -> orchestrator fetch	3 ms	fetch.min.bytes=1, fetch.max.wait.ms=5 (P0/P1), long-poll, rack-aware fetch-from-follower disabled for P0
Plan + DB commit	8 ms	In-memory per-tenant catalog and tenant cache, SET LOCAL app.tenant in the same batch, pgx batch pipeline (one round trip), prepared statements, NVMe, synchronous_commit=on with group commit
Fast-publish task	3 ms	Publish right after COMMIT from the same goroutine; relay only as fallback
Adapter fetch + dispatch	4 ms	Key-lane workers, warm HTTP/2 pools to NFs, OAuth2 token cache, pre-resolved NRF discovery
Total platform overhead	<= 25 ms	Excludes NE processing time

5.10 External network coverage (modern protocols only)

Domain	Network function / system	Standard / interface	Protocol
5G Core (SA, 5G-Advanced)	UDM / UDR subscription data (auth, AM, SMF selection, SM, SMS, LCS, slice)	3GPP TS 29.505 (UDR data model), TS 29.504 Nudr_DR, TS 29.503 Nudm	HTTP/2 + JSON (SBA), OAuth2 via NRF (TS 29.510), SCP indirect comms
5G Core	PCF policy data	TS 29.519 (policy data in UDR), TS 29.507/29.512	HTTP/2 SBA
5G Core	Network slicing (NSSF, NSSAI entitlements), NSACF quotas	TS 29.531, TS 29.536, TS 28.541 NRM	HTTP/2 SBA; 28.532 MnS REST
5G Core	SMS over NAS	TS 29.540 Nsmsf; SMS subscription in UDR	HTTP/2 SBA
5G Core / exposure	NEF (IoT, QoS, device triggering), AF influence	TS 29.522 Nnef, TS 29.122 T8	HTTP/2 / HTTPS REST
4G EPC (NSA)	HSS/UDM converged via vendor Provisioning Gateway	Vendor REST (OpenAPI), 3GPP UDR shared data	HTTPS REST/JSON (no MML)
IMS / VoLTE / VoNR / Wi-Fi Calling	IMS HSS, TAS/MMTel, SBC config, ENUM	Vendor PG REST, TS 29.562 (Nhss_ims), ENUM via REST/DNS-API	HTTP/2 SBA / HTTPS REST
Charging	CHF / OCS account, balance, bundles	TMF654, TMF666, TS 32.291 (Nchf) for policy counters	HTTPS REST
Number portability	National NPDB / clearing house	Regulator REST API	HTTPS REST
eSIM (consumer + IoT)	SM-DP+ / eIM	GSMA SGP.22 v3 ES2+ (DownloadOrder, ConfirmOrder, CancelOrder, ReleaseProfile), SGP.32 (IoT eSIM, ES2+/eIM)	HTTPS REST/JSON, mTLS
Transport / IP / Core routers	Routers, SDN controllers	IETF NETCONF RFC 6241, RESTCONF RFC 8040, YANG 1.1 RFC 7950, L3SM RFC 8299, L2SM RFC 8466, OpenConfig gNMI	NETCONF over SSH/TLS, RESTCONF HTTPS, gNMI gRPC
RAN	gNB / O-RAN O-DU/O-CU, SMO	O-RAN O1 (NETCONF/YANG), 3GPP TS 28.532 provMnS	NETCONF, HTTPS REST
Fixed broadband / FTTH	OLT/ONU, BNG, CPE	BBF TR-385/TR-383 (YANG), TR-459 (disaggregated BNG), TR-369 USP (CPE)	NETCONF, USP over MQTT/WebSocket/STOMP
Network APIs (Open Gateway)	CAMARA APIs (QoD, SIM Swap, Number Verification, Device Status)	GSMA Open Gateway / CAMARA	HTTPS REST, OIDC/CIBA
OSS / BSS	Inventory, alarms, CRM	TMF638/639/642/688	HTTPS REST / Kafka

Excluded by design: MML over Telnet, X.25/X.29, async serial, FIFO/rcp, CORBA IIOP, SOAP RPC/encoded, SNMPv1/v2c for configuration, TR-069 (CWMP, replaced by TR-369 USP) and SMPP for provisioning. Any NE that only speaks these must be upgraded or replaced, or reached through its vendor's modern northbound API, before cut-over (see the migration section).

5.11 Deployment architecture

- Kubernetes (3 AZs). Go services as Deployments with PodDisruptionBudgets and topology spread constraints.
- Kafka 4.x in KRaft mode (no ZooKeeper) via the Strimzi operator: 6-9 brokers on NVMe, 3 controllers, rack awareness, RF=3, min.insync.replicas=2, tiered storage (KIP-405) for long retention.
- PostgreSQL 18+ via CloudNativePG or Patroni: primary + 1 synchronous standby (other AZ) + async replicas, PgBouncer in transaction mode, tenant LIST partitions. When one primary is not enough, Citus distributes by tenant (the multi-tenant model Citus is built for).
- Tenant placement: tenants share the pool by default. A premium MVNO can get dedicated Kafka topics, dedicated PostgreSQL partitions, or a dedicated DxPS cell (separate namespace + database) with the same code and schema.
- Schema Registry (Apicurio or Confluent) with protobuf, BACKWARD_TRANSITIVE compatibility.
- DR: active/passive region. MirrorMaker 2 (or cluster linking) for topics including consumer offsets; PostgreSQL streaming replication; RPO <= 5 s; RTO <= 15 min.
- Autoscaling: KEDA Kafka scaler on consumer lag per tier/domain; HPA on CPU for the gateway.

5.12 Security

- mTLS between all services (service mesh or SPIFFE/SPIRE identities); TLS 1.3 to external NEs.
- Northbound: OAuth2 / OIDC (client credentials). Each tenant has its own OAuth2 clients; tenant comes from the token claim, scopes are per TMF API and per command spec, and rate limits are per tenant and client.

- Tenant isolation in PostgreSQL: row-level security on every table (policy tenant = current_setting('app.tenant')), FORCE ROW LEVEL SECURITY, and an application role without BYPASSRLS. Only the audited ops role can work across tenants.
- Tenant isolation tests run in CI: every API and query is executed as tenant A against tenant B data, and any row returned fails the build.
- Per-tenant NE credentials: when an MVNO owns NEs, its credentials sit under a tenant-scoped Vault path (dxps/<tenant>/...).
- 5G SBA: OAuth2 access tokens from NRF (TS 33.501 / TS 29.510), token caching, SEPP for roaming exposure.
- Kafka: SASL/OAUTHBEARER or mTLS, ACLs per topic and consumer group, encryption at rest.
- Secrets in HashiCorp Vault / cloud KMS. No credentials in configs (the legacy engine keeps encrypted passwords in flat files).
- PII minimisation: encrypt sensitive params (Ki/OPc are never stored; reference HSM/key-store handles only), mask in logs, audit trail table + immutable log sink.

5.13 Observability and operations

- OpenTelemetry traces: W3C traceparent in Kafka record headers, propagated to NE HTTP calls. One trace per order across all services.
- Metrics (Prometheus): per-tier and per-tenant lag, end-to-end latency histograms, NE latency/error by NE, operation and tenant, circuit state, compensation rate, DLQ depth, quota rejections per tenant.
- Logs: structured JSON (slog), correlation ids tenant, order_id, bc_id, task_id, entity_key.
- Per-tenant SLA reports (latency, success rate, volume) for MVNO billing and contracts.
- SLO dashboards and alerting (TMF642 alarms for business-impacting failures).
- Ops tooling: DLQ replay, order re-drive, task skip/force-complete with dual approval and audit.

5.14 Operations and configuration console

The DxPS console is the human interface for running and configuring the platform. The Mosaic dashboard stays as a read-only live view; the console adds sign-in, roles and changes. It sees all tenants, and every change it makes is written to the DxPS audit trail with actor console:<user>.

Area	Capabilities
Operations	Dashboard of health, rates and lanes; service orders with the task graph and the NE request and response of every attempt; cancel; NE tasks; dead letters with redrive; simulator fault injection; Kafka topics and consumer lag
Configuration	Service catalog browser; catalog studio (drafts, validate, export); Template Lab (dry run of a spec through the real planner and renderers, no NE called); order composer; tenants (SLA tier, weight, quota, status); NE inventory (limits and entitlements)
Insight	Reports (volume, success, latency by tenant, spec and NE); load runs and security probes; test results; webhooks; audit trail
Administration	Console users and roles, settings, own password

Role	Adds
viewer	Read access to every page
operator	Submit and cancel orders, NE faults, DLQ redrive, load runs and probes
configurator	Tenant and NE inventory edits, catalog drafts, Template Lab
admin	Console user management

- Catalog changes are never applied to running services directly: a draft is validated (the full catalog is loaded with the change and every spec example is dry-run), tried in the Lab, exported, reviewed and released with the next build, so the catalog stays under version control.
- Tenant, NE and entitlement changes apply live: the services reload the registry within about 10 seconds.

- Orders entered in the console use the same northbound path as any client: the gateway with a short-lived tenant token, so quotas, validation and idempotency apply unchanged.

6. Low Level Design

6.1 Go code base layout

```
dxps/
  cmd/
    gateway/           # TMF641/652/640 REST + gRPC -> Kafka
    orchestrator/      # BC + result consumers, sequencer, planner, saga
    adapter/           # one binary, domain selected by config (sba|ims|netconf|esim|bss|access|exposure)
    eventhub/          # TMF688 hub + webhook delivery
    outbox-relay/      # fallback publisher
    catalogsvc/        # TMF633/634 catalog API (tenant overrides + GLOBAL defaults)
    tenantsvc/         # tenant registry, onboarding, quotas, NE entitlements
  internal/
    tenant/            # tenant context (ctx key), token claim extraction, quota buckets, DRR fairness
    kafka/             # franz-go wrappers: producer, tiered consumer, key-lane executor, EOS helpers
    store/             # pgx v5 + sqlc generated queries, migrations (goose / atlas)
    sequencer/         # entity admission control
    planner/           # catalog -> DAG, coalescing, CEL evaluation
    saga/              # state machines, compensation
    scheduler/         # weighted-fair priority scheduler with aging
    adapters/spi/      # Adapter interface, NE registry, rate limiting, circuit breaker
    adapters/sba/      # 3GPP Nudr / Nudm / Npcf clients (oapi-codegen from 3GPP OpenAPI YAML)
    adapters/netconf/  # NETCONF (nemith/netconf) + RESTCONF + gNMI (openconfig/gnmi)
    adapters/esim/     # GSMA ES2+ client
    adapters/...       # ims, bss, access (USP), exposure (CAMARA/NEF)
    tmf/               # TMF641/652/688 models (generated from TMF OpenAPI)
    observability/     # OTel, slog, Prometheus
  api/proto/           # protobuf contracts (BusinessCommand, NeTask, TaskResult, OrderEvent)
  api/openapi/         # TMF + 3GPP OpenAPI specs
  deploy/helm/  deploy/kafka/  deploy/postgres/
```

Concern	Library / choice	Reason
Kafka client	github.com/twmb/franz-go (kgo, kadm)	Pure Go, high throughput, full EOS/transactions, KIP-848, pause/resume, low allocation
PostgreSQL	github.com/jackc/pgx/v5 + sqlc	Binary protocol, batch/pipeline, COPY, type-safe generated queries
HTTP/2 clients	net/http + golang.org/x/net/http2 (h2 and h2c)	3GPP SBA requires HTTP/2; connection multiplexing
gRPC	google.golang.org/grpc / connectrpc	Internal APIs, gNMI
Rules	github.com/google/cel-go	Safe, fast, sandboxed decomposition and validation expressions
Resilience	golang.org/x/time/rate, sony/gobreaker	Per-NE token bucket and circuit breaker
Codegen	oapi-codegen (3GPP/TMF OpenAPI), buf (protobuf)	Contract-first
Observability	go.opentelemetry.io/otel, log/slog, prometheus/client_golang	Standard telemetry
Runtime tuning	Container-aware GOMAXPROCS (Go 1.25+), GOMEMLIMIT, PGO builds	Lower GC pauses and better tail latency

6.2 Message contracts (protobuf)

```
syntax = "proto3";
package dxps.v1;
import "google/protobuf/timestamp.proto";
import "google/protobuf/struct.proto";

enum Priority { P_UNSPECIFIED = 0; P0 = 1; P1 = 2; P2 = 3; P3 = 4; }
enum TxMode   { ATOMIC = 0; PARTIAL_ALLOWED = 1; BEST_EFFORT = 2; }

message BusinessCommand {                  // topic dxps.bc.p{0..3}, key = tenant|entity_key
  string tenant       = 1;               // max 40 chars; set by gateway from the OAuth2 token
  string message_id      = 2;            // UUIDv7, producer-generated (dedupe)
  string order_id        = 3;            // TMF641 ServiceOrder.id (uuid)
  string order_item_id   = 4;            // ServiceOrderItem.id
  string bc_id           = 5;            // uuid
```

```
  string entity_key      = 6;              // e.g. "supi:imsi-4150..." or "msisdn:9627..."
  int64  entity_seq      = 7;              // monotonic per (tenant, entity_key) at source
  repeated string extra_entity_keys = 8;   // multi-entity commands (same tenant)
  string command_spec    = 9;              // catalog id, e.g. "CreateSubscriber5G"
  string spec_version    = 10;
  string action          = 11;             // add | modify | delete | noChange (TMF641 itemAction)
  Priority priority      = 12;
  TxMode tx_mode         = 13;
  google.protobuf.Struct params = 14;      // validated against the tenant's catalog JSON Schema
  google.protobuf.Timestamp requested_start = 15;
  google.protobuf.Timestamp deadline = 16;
  string channel         = 17;             // CRM, APP, RETAIL, B2B, MVNO_BSS, MIGRATION
}

message NeTask {                           // topic dxps.task.<domain>.p{0..3}, key = tenant|ne_id|entity_key
  string tenant = 1; string task_id = 2; string order_id = 3; string ne_id = 4; string entity_key = 5;
  string operation = 6;                    // e.g. "udr.authSubscription.put"
  bytes  request = 7;                      // rendered payload (JSON / XML-YANG / protobuf)
  string idempotency_key = 8;              // = tenant + ":" + task_id + ":" + attempt_generation
  int32  attempt = 9; Priority priority = 10;
  google.protobuf.Timestamp not_before = 11; google.protobuf.Timestamp deadline = 12;
  bool   compensation = 13;
}

message TaskResult {                       // topic dxps.task.result, key = tenant|order_id
  string tenant = 1; string task_id = 2; string order_id = 3;
  enum Outcome { SUCCEEDED = 0; RETRYABLE = 1; FAILED = 2; }
  Outcome outcome = 4;
  int32 ne_status = 5; string ne_code = 6; string message = 7;
  bytes  response = 8;                     // normalised; used for downstream params (e.g. returned IDs)
  int64  latency_us = 9;
}
```

Kafka record headers: tenant, traceparent, tracestate, message_id, priority, schema_id, not_before (retry topics), producer_app. Consumers reject any record whose tenant header and payload field differ. Header values are small ASCII so they are cheap to read without deserialising the payload.

6.3 Kafka client and broker configuration

6.3.1 Producer (franz-go equivalents)

Setting	P0 / P1	P2 / P3	Rationale
acks	all	all	Zero data loss with min.insync.replicas=2
enable.idempotence	true	true	No duplicates / reordering on retry (default in franz-go)
max.in.flight.requests.per.connection	5	5	Max allowed with idempotence ordering guarantee
linger.ms (kgo.ProducerLinger)	0-1 ms	5-20 ms	Latency vs batching trade-off per tier
batch.size / ProducerBatchMaxBytes	64 KiB	512 KiB - 1 MiB	Bigger batches for bulk
compression	lz4	zstd	lz4 = lowest CPU latency; zstd = best ratio for bulk
partitioner	murmur2 on key (Java-compatible)	same	Cross-language key placement consistency (kgo.StickyKeyPartitioner with Kafka hasher)
delivery.timeout.ms / RecordDeliveryTimeout	5 s	120 s	Fail fast for interactive tiers
transactional.id	orchestrator/adapter: <svc>-<partition-owner>	same	Only for consume-transform-produce paths

6.3.2 Consumer

Setting	Value	Rationale
group.protocol	consumer (KIP-848, Kafka 4.x)	Server-side incremental assignment; faster, lower-impact rebalances
group.instance.id	pod name (StatefulSet ordinal or stable id)	Static membership: rolling restarts do not trigger

		rebalances
isolation.level	read_committed	Ignore aborted transactional writes
enable.auto.commit	false	Commit only contiguous completed offsets
fetch.min.bytes / fetch.max.wait.ms	1 / 5 ms (P0,P1); 64 KiB / 50 ms (P3)	Latency vs efficiency per tier
max.poll.records (PollRecords n)	500 (P0/P1), 2000 (P3)	Bounded in-flight
max.partition.fetch.bytes	1 MiB	Avoid head-of-line from big batches
client.rack	AZ id	Rack-aware assignment; fetch-from-follower only for P3 to save cross-AZ cost

6.3.3 Broker / topic

Setting	Value	Rationale
replication.factor / min.insync.replicas	3 / 2	Survive one broker/AZ loss with no data loss
unclean.leader.election.enable	false	Never lose acknowledged data
message.timestamp.type	CreateTime	Latency measurement end-to-end
compression.type (topic)	producer	No broker recompression
segment.ms / segment.bytes	1 h / 256 MiB	Fast retention and compaction cycles
cleanup.policy (state topics)	compact, min.compaction.lag.ms=60000	Latest snapshot per key
num.replica.fetchers / num.network.threads / num.io.threads	4 / 8 / 16	NVMe-backed low-latency brokers
remote.storage.enable (KIP-405)	true for bc and event topics	Cheap long retention without broker disk growth
Share groups (KIP-932, Queues for Kafka)	Evaluate for P3 bulk once GA	Per-record acks and parallelism beyond partition count where ordering is not required
Client quotas (producer_byte_rate / consumer_byte_rate / request_percentage)	Per tenant principal (native producers, dedicated topics)	Noisy-neighbour protection at broker level
Prefixed ACLs	dxps.t.<tenant>. -> tenant principal only	Isolation for dedicated MVNO topics

6.4 Tiered consumer and weighted-fair scheduler (Go)

```
// One kgo.Client (consumer group) per tier; all feed one Scheduler.
type Tier int
const (P0 Tier = iota; P1; P2; P3)
var weights = [4]int{8, 4, 2, 1}

type Scheduler struct {
    q       [4]chan *kgo.Record       // bounded per tier (e.g. 4096)
    clients [4]*kgo.Client
    topics  [4]string
}

func (s *Scheduler) pollTier(ctx context.Context, t Tier) {
    cl := s.clients[t]
    for ctx.Err() == nil {
        fs := cl.PollRecords(ctx, 500)
        fs.EachRecord(func(r *kgo.Record) {
            select {
            case s.q[t] <- r:
            default: // backpressure: queue full
                cl.PauseFetchTopics(s.topics[t])
                s.q[t] <- r                                  // block until space
                if len(s.q[t]) < cap(s.q[t])/2 { cl.ResumeFetchTopics(s.topics[t]) }
            }
        })
    }
}

// Weighted round-robin with strict P0 pre-emption and aging.
func (s *Scheduler) Next(ctx context.Context) *kgo.Record {
    for {
        select { case r := <-s.q[P0]: return r; default: }            // strict for P0
        for t := P1; t <= P3; t++ {
            for i := 0; i < weights[t]; i++ {
```

```
                select {
                case r := <-s.q[t]:
                    if t > P1 && aged(r) { metrics.Promoted.Inc() }    // aging accounted in weights
                    return r
                default:
                }
            }
        }
        select {                                                      // nothing ready: block on any
        case r := <-s.q[P0]: return r
        case r := <-s.q[P1]: return r
        case r := <-s.q[P2]: return r
        case r := <-s.q[P3]: return r
        case <-ctx.Done(): return nil
        }
    }
}
```

Tenant fairness inside a tier: each tier queue is a set of per-tenant sub-queues served by deficit round robin. A tenant's quantum comes from its SLA tier, so a bulk load from one MVNO is interleaved with other tenants instead of blocking them. Per-entity order is kept because one tenant's entity always lands in that tenant's sub-queue in arrival order.

```
type tenantQueue struct { id string; q []*kgo.Record; deficit, quantum int }

// Deficit round robin over the tenants of one tier (unit cost per record).
// quantum = tenant SLA weight, e.g. HOST 8, FULL_MVNO 4, LIGHT_MVNO 2.
func (tq *tierQueue) next() *kgo.Record {
    for i := 0; i < len(tq.active); i++ {
        t := tq.active[tq.cursor]
        if len(t.q) == 0 {                          // idle tenant loses its credit
            t.deficit = 0
            tq.cursor = (tq.cursor + 1) % len(tq.active)
            continue
        }
        if t.deficit == 0 { t.deficit = t.quantum }   // new turn
        r := t.q[0]; t.q = t.q[1:]; t.deficit--
        if t.deficit == 0 { tq.cursor = (tq.cursor + 1) % len(tq.active) }
        return r
    }
    return nil
}
```

6.5 Key-lane executor (ordering with parallelism)

A partition holds many keys. Processing it serially caps throughput; processing it fully in parallel breaks order. The key-lane executor runs records with the same key serially and different keys in parallel, and commits only the highest contiguous completed offset per partition.

```
type lane struct{ ch chan *kgo.Record }

type KeyLaneExec struct {
    lanes    sync.Map                        // key -> *lane (evicted when idle)
    tracker  map[topicPartition]*offsetTracker // contiguous-completion tracking
    sem      chan struct{}                   // global concurrency cap (e.g. 2048)
}

func (e *KeyLaneExec) Submit(r *kgo.Record, h func(context.Context, *kgo.Record) error) {
    e.trackerFor(r).Begin(r.Offset)
    l := e.laneFor(string(r.Key))             // same key -> same goroutine, FIFO
    l.ch <- r
    // lane goroutine: for r := range l.ch { e.sem <- struct{}{}; err := h(ctx, r); <-e.sem; e.done(r, err) }
}

func (e *KeyLaneExec) done(r *kgo.Record, err error) {
```

```
    if err != nil { routeToRetryOrDLQ(r, err) }       // never block the lane on a poison message
    if off, ok := e.trackerFor(r).Complete(r.Offset); ok {
        e.commitQueue <- kgo.EpochOffset{Offset: off + 1} // committed in batches every 50 ms or 1k recs
    }
}
```

6.6 Orchestrator: BC ingestion step

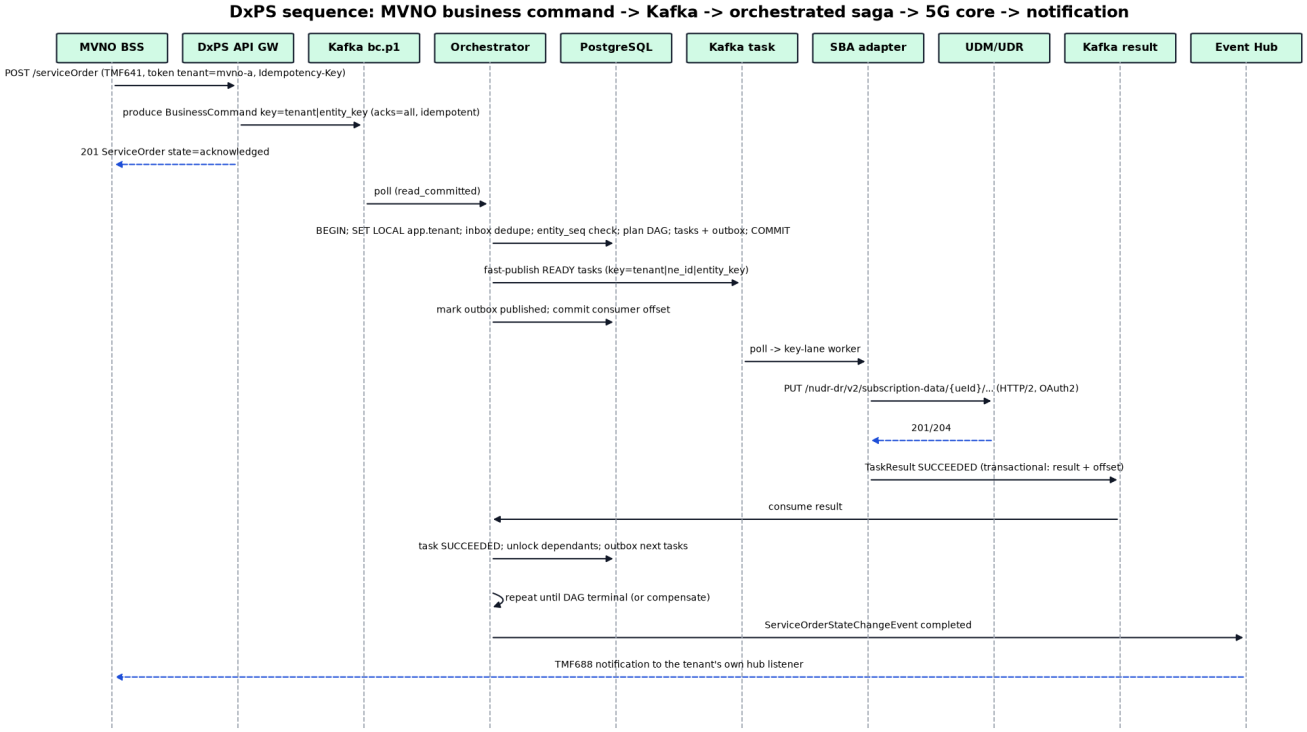


Figure 5 - Activation sequence (happy path)

```
func (o *Orchestrator) HandleBC(ctx context.Context, rec *kgo.Record) error {
    bc := decodeBC(rec)                                  // protobuf, schema id from header
    ten, ok := o.tenants.Get(bc.Tenant)                 // in-memory, fed by compacted dxps.state.tenant
    if !ok || ten.Status != Active || header(rec, "tenant") != bc.Tenant {
        return o.reject(ctx, bc, "DXPS-1004", errTenant)
    }
    spec, err := o.catalog.Resolve(bc.Tenant, bc.CommandSpec, bc.SpecVersion) // tenant override, then GLOBAL
    if err != nil { return o.reject(ctx, bc, "DXPS-1001", err) }
    if err := spec.Validate(bc.Params); err != nil { return o.reject(ctx, bc, "DXPS-1002", err) }

    var toPublish []outboxRow
    err = pgx.BeginTxFunc(ctx, o.db, pgx.TxOptions{IsoLevel: pgx.ReadCommitted}, func(tx pgx.Tx) error {
        // 0. Tenant context for row-level security (transaction-scoped, safe with PgBouncer)
        if _, err := tx.Exec(ctx, "SELECT set_config('app.tenant', $1, true)", bc.Tenant); err != nil { return err }
        // 1. Inbox dedupe (exactly-once processing): PK (tenant, id = message_id)
        if dup, err := q.InboxInsert(ctx, tx, bc.Tenant, bc.MessageId); err != nil || dup { return err }
        // 2. Persist order/BC (upsert order, insert BC)
        if err := q.UpsertOrderAndBC(ctx, tx, bc); err != nil { return err }
        // 3. Entity admission (row lock on entity_sequencer only - short critical section)
        admitted, err := o.seq.Admit(ctx, tx, bc)        // FOR UPDATE on (tenant, entity_key) rows (sorted)
        if err != nil || !admitted { return err }        // parked: released when predecessor terminal
        // 4. Plan DAG (pure, in-memory) with coalescing against other admitted BCs of the order
        //    NE selection limited to NEs the tenant owns or is entitled to (tenant_ne_access)
        plan := o.planner.Build(ten, spec, bc, q.PendingSiblings(ctx, tx, bc.Tenant, bc.OrderId))
        // 5. Insert tasks, links (M:N), edges; ready roots go to outbox
        toPublish, err = q.InsertPlan(ctx, tx, plan)     // pgx.Batch: single round trip
        return err
    })
```

```
    if err != nil { return err }                        // record retried by the executor; dedupe makes it safe
    o.fastPublish(ctx, toPublish)                        // ProduceSync, then mark published; relay covers crashes
    return nil
}
```

6.7 Entity sequencer

```
-- Admission: called inside the BC transaction ($1 = tenant, $2 = entity keys)
INSERT INTO entity_sequencer (tenant, id, entity_key)
SELECT $1, uuidv7(), k FROM unnest($2::text[]) k
ON CONFLICT (tenant, entity_key) DO NOTHING;                          -- first command for an entity

SELECT id, entity_key, next_seq, active_bc_id FROM entity_sequencer
 WHERE tenant = $1 AND entity_key = ANY($2::text[])
 ORDER BY entity_key FOR UPDATE;                                          -- canonical order => no deadlock

-- admitted if for every key: active_bc_id IS NULL AND bc.entity_seq = next_seq
--            (or spec.preempt = true -> supersede rules applied to parked BCs)
UPDATE entity_sequencer SET active_bc_id = $3, updated_at = now()
 WHERE tenant = $1 AND entity_key = ANY($2);

-- On BC terminal (completed / failed / cancelled):
UPDATE entity_sequencer SET active_bc_id = NULL, next_seq = next_seq + 1, updated_at = now()
 WHERE tenant = $1 AND entity_key = ANY($2);
-- then release the next parked BC (tenant = $1, state = 'pending', entity_seq = next_seq), in the same tx
```

Gap handling: if entity_seq n+1 arrives and n never arrives within gap_timeout (default 30 s, per spec), the sequencer raises a TMF642 alarm. Depending on policy it either holds the BC (TMF641 state held) or skips the gap. Sources that cannot supply entity_seq use 0, which means arrival order within the partition.

6.8 Result handling, retries, compensation

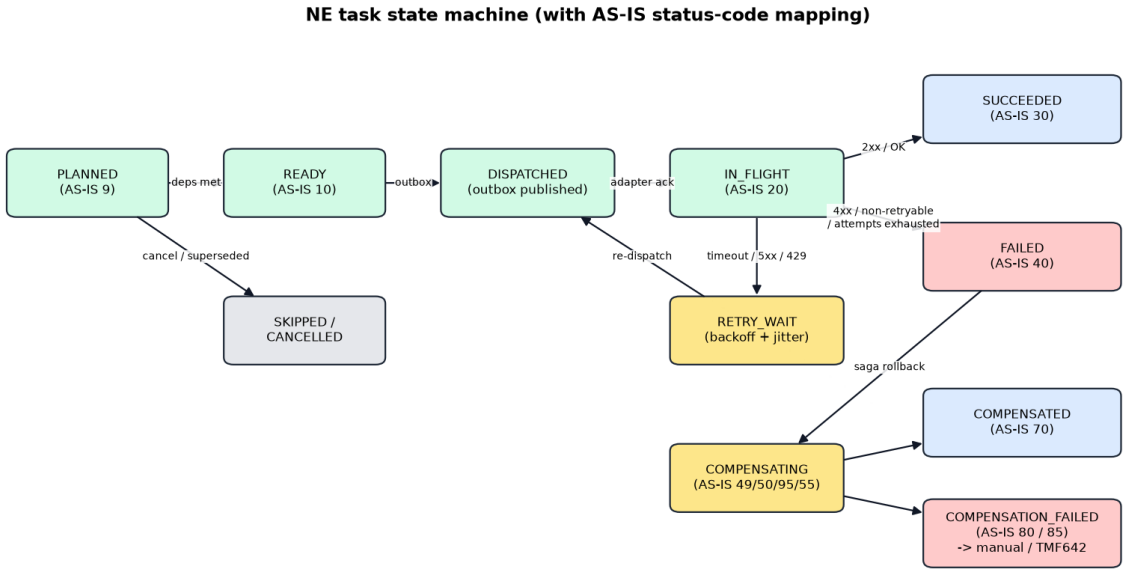


Figure 6 - NE task state machine

Outcome	Classification	Action
2xx / NETCONF <ok/> / gNMI OK	SUCCEEDED	Mark task, merge response params, unlock dependants (in-degree 0 -> READY -> outbox)
408, 429, 500, 502, 503, 504, connect/timeout, NRF/SCP 503	RETRYABLE	attempt++, exponential backoff with full jitter (base 200 ms, cap 5 min) via dxps.retry.* ladder; honour Retry-After
400, 403, 404 (on update), 409 conflict, YANG validation error	FAILED	No retry. ATOMIC -> start compensation; PARTIAL -> item failed, others continue
409 on create where resource exists with same content	SUCCEEDED (idempotent)	Read-back compare; treat as success

Attempts exhausted (default 8) or deadline passed	FAILED	As above; DLQ copy for ops
Circuit open for NE	RETRYABLE (no call made)	Route to retry topic; do not consume NE capacity

```
func (o *Orchestrator) HandleResult(ctx context.Context, r *TaskResult) error {
    return pgx.BeginTxFunc(ctx, o.db, pgx.TxOptions{}, func(tx pgx.Tx) error {
        setTenant(ctx, tx, r.Tenant)                  // set_config('app.tenant', ..., true) for RLS
        ord := q.LockOrder(ctx, tx, r.Tenant, r.OrderId) // SELECT ... WHERE (tenant, id) = ... FOR UPDATE
        t := q.UpdateTaskOutcome(ctx, tx, r)
        switch t.State {
        case Succeeded:
            for _, d := range q.DecrementInDegree(ctx, tx, r.Tenant, t.ID) { if d.InDegree == 0 {
q.ReadyAndOutbox(ctx, tx, d) } }
        case Failed:
            if ord.Mode(t) == ATOMIC && !t.AfterPivot { q.PlanCompensation(ctx, tx, ord) } // reverse topo order
            else { q.MarkItemFailed(ctx, tx, t) }
        }
        if q.OrderTerminal(ctx, tx, ord) {
            q.ReleaseSequencer(ctx, tx, ord)             // admits next parked BC per entity
            q.OutboxEvent(ctx, tx, tmf688.StateChange(ord))
        }
        return nil
    })
}
```

6.9 Adapter SPI

```
type Adapter interface {
    Domain() string                                                  // "sba", "netconf", "esim", ...
    Supports(op string) bool
    Execute(ctx context.Context, ne NE, t *dxpsv1.NeTask) (Result, error) // must be idempotent per t.IdempotencyKey
    Compensate(ctx context.Context, ne NE, t *dxpsv1.NeTask) (Result, error)
    Health(ctx context.Context, ne NE) HealthStatus
}

type NE struct {
    Tenant, ID string        // owner tenant (HOST or FULL_MVNO) + uuid
    Code, Domain, Vendor, NFType, Release string
    Shared    bool
    Endpoints []Endpoint      // weighted, health-checked, geo-redundant (N:M with adapters)
    Limiter   *rate.Limiter   // global max_tps; separate reserved bucket for P0/P1
    TenantLim map[string]*rate.Limiter // per-tenant quota from tenant_ne_access
    Conc      *semaphore.Weighted
    Breaker   *gobreaker.CircuitBreaker
}

// Acquire applies both the NE-wide and the calling tenant's budget before any NE call.
func (ne *NE) Acquire(ctx context.Context, tenant string, p Priority) error {
    if l, ok := ne.TenantLim[tenant]; ok { if err := l.Wait(ctx); err != nil { return err } }
    else if ne.Tenant != tenant { return errNotEntitled }   // DXPS-1005
    return ne.Limiter.Wait(ctx)
}
```

Adapter	Key operations	Implementation notes
sba (5G core)	UDR provisioned data PUT/PATCH/DELETE (authentication-subscription, am-data, smf-selection-subscription-data, sm-data, sms-data), PCF policy data, NSSF/NSACF	HTTP/2 (h2 with TLS 1.3), JSON Merge Patch / JSON Patch (RFC 7396/6902), OAuth2 token cache from NRF, NRF discovery cache with TTL, SCP Model C/D, 3gpp-Sbi-* headers
ims	IMPI/IMPU, IFC/service profile, TAS MMTel supplementary services, ENUM NAPTR	Vendor PG REST/OpenAPI; ENUM through DNS provider REST API
netconf	edit-config on candidate, validate, confirmed-commit, commit; RESTCONF PATCH (YANG-Patch RFC 8072); gNMI Set	Session pool per NE over SSH/TLS; YANG models compiled with ygot; rollback-on-error
esim	ES2+ DownloadOrder, ConfirmOrder, CancelOrder, ReleaseProfile, HandleDownloadProgressInfo (callback)	mTLS with GSMA certificates; async callback mapped to TaskResult
bss	OCS/CHF account and bundle, TMF654/666, NPDB	REST; idempotency via request id

	porting	
access	OLT/ONT service (TR-385 YANG), CPE (TR-369 USP Set/Add/Operate)	NETCONF for OLT; USP controller over MQTT 5 / WebSocket
exposure	NEF/CAMARA subscriptions, QoD profiles	HTTPS REST, OIDC client credentials

6.10 Catalog / decomposition (replaces the legacy MMLGEN templates)

Specs live in command_spec with a tenant. tenant = 'GLOBAL' holds the default spec. An MVNO can publish its own version of a spec (for example a different OCS step), and resolution tries the tenant first, then GLOBAL.

```
tenant: GLOBAL                # or a tenant, e.g. mvno-alpha, to override for that MVNO only
spec: CreateSubscriber5G
version: 3.2.0
txMode: ATOMIC
priorityDefault: P1
preempt: false
params:                       # JSON Schema (validated in gateway and orchestrator)
  required: [supi, msisdn, plan]
  properties:
    supi:   {type: string, pattern: "^imsi-[0-9]{15}$"}
    msisdn: {type: string, pattern: "^[0-9]{8,15}$"}
    plan:   {type: string}
entityKeys: ["'supi:' + params.supi", "'msisdn:' + params.msisdn"]   # CEL
tasks:
  - id: auth
    ne: "neSelect(tenant, 'UDR', params.supi)"  # CEL: only NEs the tenant owns or is entitled to; range/region rules
    op: udr.authSubscription.put
    template: udr/auth-subscription.json.tmpl
    compensate: udr.authSubscription.delete
  - id: am
    dependsOn: [auth]
    op: udr.amData.put
    coalesceKey: "'udr-am:' + params.supi"      # merges with other BCs touching the same resource
    template: udr/am-data.json.tmpl           # includes tenant subscriber group / S-NSSAI / DNN on shared UDR
    compensate: udr.amData.delete
  - id: policy
    dependsOn: [auth]
    when: "params.plan.startsWith('5G')"
    op: udr.policyData.put
    compensate: udr.policyData.delete
  - id: ocs
    dependsOn: [am]
    op: bss.account.create
    pivot: true                                  # no compensation after this point
```

6.11 PostgreSQL physical design (multi-tenant)

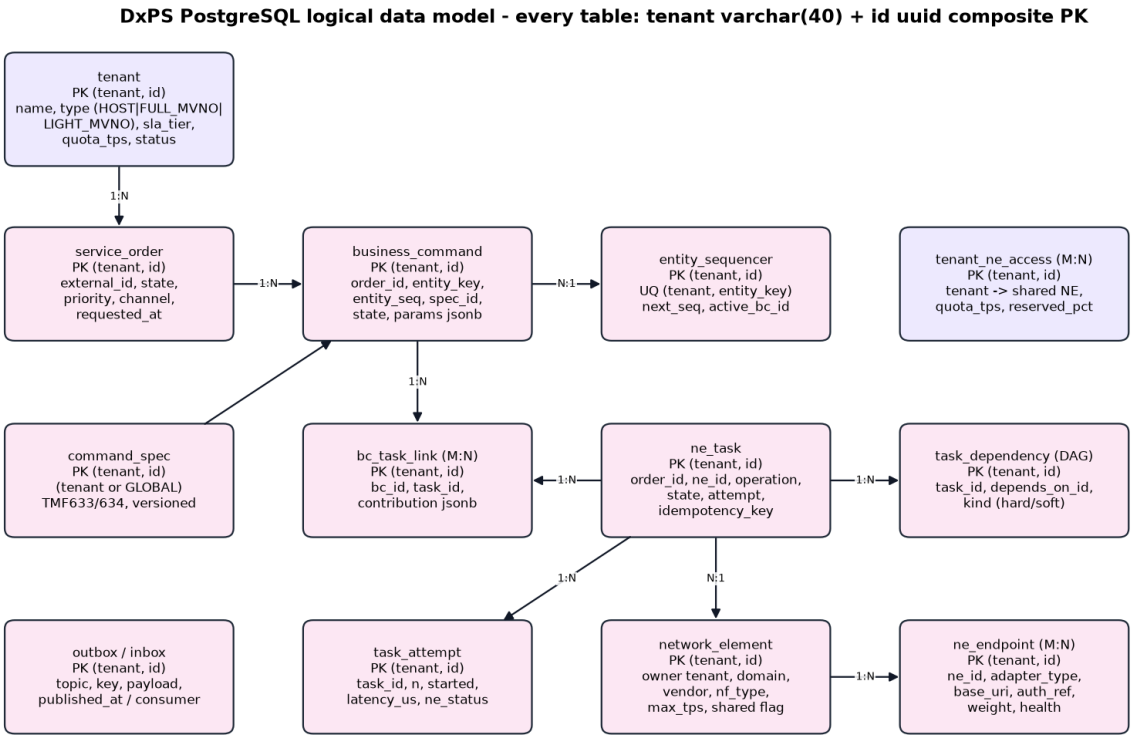


Figure 7 - DxPS core data model

Mandatory rules for every DxPS table:

- Column tenant varchar(40) NOT NULL (domain dxps_tenant, pattern ^[A-Za-z0-9_.-]{1,40}$) is the first column.
- Column id uuid NOT NULL (UUIDv7 from uuidv7() in PostgreSQL 18, time-ordered for B-tree locality).
- PRIMARY KEY (tenant, id). Natural keys become UNIQUE (tenant, ...).
- Foreign keys are composite: (tenant, parent_id) REFERENCES parent (tenant, id). A row can never point at another tenant's row.
- Row-level security is enabled and forced, with the tenant_isolation policy below.
- Every index starts with tenant, so lookups are tenant-scoped and partition-prunable.

```
CREATE DOMAIN dxps_tenant AS varchar(40)
  CHECK (VALUE ~ '^[A-Za-z0-9_.-]{1,40}$');

-- ---------------------------------------------------------------- tenancy
CREATE TABLE tenant (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  name text NOT NULL, tenant_type text NOT NULL,            -- HOST | FULL_MVNO | LIGHT_MVNO | ENTERPRISE
  host_tenant dxps_tenant,                              -- MVNO -> its host MNO
  sla_tier text NOT NULL, sla_weight smallint NOT NULL,     -- DRR quantum
  quota_tps int NOT NULL, status text NOT NULL,             -- ACTIVE | SUSPENDED | OFFBOARDING
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant, id), UNIQUE (tenant)
);

CREATE TABLE network_element (                              -- tenant = owning tenant (HOST or FULL_MVNO)
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  code text NOT NULL, domain text NOT NULL, vendor text, nf_type text NOT NULL, release text,
```

```
  shared bool NOT NULL DEFAULT false, max_tps int NOT NULL, max_concurrency int NOT NULL,
  reserved_pct smallint NOT NULL DEFAULT 20, attrs jsonb,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, code)
);

CREATE TABLE ne_endpoint (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  ne_id uuid NOT NULL, adapter_type text NOT NULL, base_uri text NOT NULL,
  auth_ref text NOT NULL,                                  -- Vault path dxps/<tenant>/...
  weight int NOT NULL DEFAULT 100, site text,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, ne_id, adapter_type, base_uri),
  FOREIGN KEY (tenant, ne_id) REFERENCES network_element (tenant, id)
);

CREATE TABLE tenant_ne_access (                            -- many-to-many: tenant <-> shared NE
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  ne_tenant dxps_tenant NOT NULL, ne_id uuid NOT NULL,  -- NE owner (usually the host MNO)
  quota_tps int NOT NULL, reserved_pct smallint NOT NULL DEFAULT 10,
  allowed_ops text[] NOT NULL,                             -- e.g. {udr.*, pcf.policyData.*}
  subscriber_group text,                                   -- tag on the shared NE (UDR group / S-NSSAI / DNN)
  PRIMARY KEY (tenant, id), UNIQUE (tenant, ne_tenant, ne_id),
  FOREIGN KEY (ne_tenant, ne_id) REFERENCES network_element (tenant, id)
);

CREATE TABLE command_spec (                                -- catalog; tenant = 'GLOBAL' for defaults
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  spec_code text NOT NULL, version text NOT NULL, status text NOT NULL, body jsonb NOT NULL,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, spec_code, version)
);

-- ------------------------------------------------------------------ orders (LIST-partitioned by tenant)
CREATE TABLE service_order (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  external_id text, channel text NOT NULL,
  state text NOT NULL,              -- TMF641:
acknowledged|inProgress|held|pending|completed|failed|partial|cancelled|...
  priority smallint NOT NULL, tx_mode text NOT NULL,
  requested_at timestamptz NOT NULL, completed_at timestamptz, version int NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, external_id)
) PARTITION BY LIST (tenant);

CREATE TABLE business_command (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  order_id uuid NOT NULL, entity_key text NOT NULL, entity_seq bigint NOT NULL,
  spec_id uuid NOT NULL, spec_tenant dxps_tenant NOT NULL,   -- own spec or GLOBAL spec
  action text NOT NULL, state text NOT NULL, priority smallint NOT NULL, params jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant, id),
  FOREIGN KEY (tenant, order_id) REFERENCES service_order (tenant, id),
  FOREIGN KEY (spec_tenant, spec_id) REFERENCES command_spec (tenant, id)
) PARTITION BY LIST (tenant);
CREATE INDEX ON business_command (tenant, order_id);
CREATE INDEX ON business_command (tenant, entity_key, entity_seq) WHERE state = 'pending';

CREATE TABLE bc_entity (                                   -- multi-entity BCs
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  bc_id uuid NOT NULL, entity_key text NOT NULL,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, entity_key, bc_id),
  FOREIGN KEY (tenant, bc_id) REFERENCES business_command (tenant, id)
) PARTITION BY LIST (tenant);

CREATE TABLE entity_sequencer (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  entity_key text NOT NULL, next_seq bigint NOT NULL DEFAULT 1,
  active_bc_id uuid, updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant, id), UNIQUE (tenant, entity_key)
) PARTITION BY LIST (tenant);                            -- fillfactor 70 on partitions (HOT updates)
```

```
CREATE TABLE ne_task (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  order_id uuid NOT NULL,
  ne_tenant dxps_tenant NOT NULL, ne_id uuid NOT NULL,  -- NE may belong to the host MNO
  operation text NOT NULL, state text NOT NULL, attempt int NOT NULL DEFAULT 0, in_degree int NOT NULL,
  priority smallint NOT NULL, is_compensation bool NOT NULL DEFAULT false, after_pivot bool NOT NULL DEFAULT false,
  idempotency_key text NOT NULL, coalesce_key text, request jsonb, response jsonb,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant, id),
  UNIQUE (tenant, idempotency_key),
  FOREIGN KEY (tenant, order_id) REFERENCES service_order (tenant, id),
  FOREIGN KEY (ne_tenant, ne_id) REFERENCES network_element (tenant, id)
) PARTITION BY LIST (tenant);                          -- fillfactor 80 on partitions
CREATE INDEX ON ne_task (tenant, order_id);
CREATE INDEX ON ne_task (tenant, order_id, coalesce_key) WHERE coalesce_key IS NOT NULL;

CREATE TABLE bc_task_link (                            -- many-to-many BC <-> task
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  order_id uuid NOT NULL, bc_id uuid NOT NULL, task_id uuid NOT NULL, contribution jsonb,
  PRIMARY KEY (tenant, id), UNIQUE (tenant, bc_id, task_id),
  FOREIGN KEY (tenant, bc_id) REFERENCES business_command (tenant, id),
  FOREIGN KEY (tenant, task_id) REFERENCES ne_task (tenant, id)
) PARTITION BY LIST (tenant);
CREATE INDEX ON bc_task_link (tenant, task_id);

CREATE TABLE task_dependency (                         -- DAG edges
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  order_id uuid NOT NULL, task_id uuid NOT NULL, depends_on_id uuid NOT NULL,
  kind text NOT NULL DEFAULT 'hard',
  PRIMARY KEY (tenant, id), UNIQUE (tenant, depends_on_id, task_id),
  FOREIGN KEY (tenant, task_id) REFERENCES ne_task (tenant, id),
  FOREIGN KEY (tenant, depends_on_id) REFERENCES ne_task (tenant, id)
) PARTITION BY LIST (tenant);

-- ------------------------------------------------------------------ append-only (RANGE on UUIDv7 id = time)
CREATE TABLE task_attempt (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  task_id uuid NOT NULL, n int NOT NULL, started_at timestamptz NOT NULL,
  latency_us bigint, ne_status int, ne_code text, error text,
  PRIMARY KEY (tenant, id)
) PARTITION BY RANGE (id);                             -- daily partitions bounded by UUIDv7 of day start; 30-day
retention
CREATE INDEX ON task_attempt (tenant, task_id);

CREATE TABLE audit_event (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  actor text NOT NULL, acting_for_tenant bool NOT NULL DEFAULT false,   -- host admin acting for an MVNO
  action text NOT NULL, object_type text NOT NULL, object_id uuid, detail jsonb,
  PRIMARY KEY (tenant, id)
) PARTITION BY RANGE (id);

-- ------------------------------------------------------------------ messaging
CREATE TABLE outbox (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  shard smallint NOT NULL,                             -- relay shard = hash(kafka key) % N
  topic text NOT NULL, key bytea NOT NULL, payload bytea NOT NULL, headers jsonb,
  created_at timestamptz NOT NULL DEFAULT now(), published_at timestamptz,
  PRIMARY KEY (tenant, id)
);
CREATE INDEX outbox_unpublished ON outbox (shard, id) WHERE published_at IS NULL;   -- relay (system role)

CREATE TABLE orchestrator_inbox (                      -- one inbox table per consuming service
  tenant dxps_tenant NOT NULL, id uuid NOT NULL,      -- id = BusinessCommand.message_id (UUIDv7)
  received_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant, id)
) PARTITION BY RANGE (id);                             -- drop partitions older than Kafka retention + 1 day
```

```
CREATE TABLE hub_subscription (                          -- TMF688 listeners per tenant
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  callback text NOT NULL, query text, secret_ref text NOT NULL, status text NOT NULL,
  PRIMARY KEY (tenant, id)
);
```

Tenant partitioning (applied to every LIST-partitioned table):

```
-- Large tenants get dedicated partitions; all others share a hash-partitioned default.
CREATE TABLE ne_task_host_mno   PARTITION OF ne_task FOR VALUES IN ('host-mno')   WITH (fillfactor = 80);
CREATE TABLE ne_task_mvno_alpha PARTITION OF ne_task FOR VALUES IN ('mvno-alpha') WITH (fillfactor = 80);
CREATE TABLE ne_task_shared     PARTITION OF ne_task DEFAULT PARTITION BY HASH (tenant);
CREATE TABLE ne_task_shared_00  PARTITION OF ne_task_shared FOR VALUES WITH (MODULUS 16, REMAINDER 0);
-- ... remainders 1..15
```

Row-level security (applied to every table):

```
ALTER TABLE ne_task ENABLE ROW LEVEL SECURITY;
ALTER TABLE ne_task FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON ne_task
  USING      (tenant = current_setting('app.tenant')::dxps_tenant)
  WITH CHECK (tenant = current_setting('app.tenant')::dxps_tenant);

-- Catalog: a tenant reads its own specs and GLOBAL defaults, and writes only its own
CREATE POLICY catalog_read  ON command_spec FOR SELECT
  USING (tenant IN (current_setting('app.tenant')::dxps_tenant, 'GLOBAL'));
CREATE POLICY catalog_write ON command_spec FOR ALL
  USING (tenant = current_setting('app.tenant')::dxps_tenant)
  WITH CHECK (tenant = current_setting('app.tenant')::dxps_tenant);

-- Roles: dxps_app (no BYPASSRLS) for services; dxps_relay / dxps_ops (BYPASSRLS, audited) for the
-- outbox relay, NE registry loader and cross-tenant operations.
```

- Every transaction sets the tenant first with set_config('app.tenant', $1, true). The setting is transaction-local, so it is safe with PgBouncer transaction pooling.
- Shared host NEs: ne_task references (ne_tenant, ne_id). PostgreSQL referential checks bypass RLS, so an MVNO task can reference a host NE, while the MVNO can still never read the host's other rows. The orchestrator gets NE entitlements from an in-memory cache loaded by dxps_relay.
- Moving a tenant from the shared default to a dedicated partition: detach the default partition, create the tenant partition, move that tenant's rows, then re-attach (a scripted, online-window procedure).
- Hot path uses only (tenant, id) primary-key access or tenant-prefixed partial indexes. No polling queries (the relay scans outbox_unpublished, which is normally empty).
- Terminal orders move to an archive schema (same keys) for reporting, so hot tables stay small.
- Settings: synchronous_commit=on with a synchronous standby (remote_write for the latency budget), wal_compression=lz4, autovacuum tuned per hot partition, PgBouncer transaction pooling (prepared statements supported in PgBouncer 1.21+).
- Horizontal scale beyond one primary: Citus distributes all tenant tables by tenant, with reference tables for GLOBAL catalog data. Composite keys and FKs already include the distribution column, so no schema change is needed.
- Tenant offboarding: export the tenant's data by tenant, then drop or detach its partitions (dedicated) or delete by tenant (shared).

6.12 Northbound API examples

TMF641 create service order from an MVNO (the gateway takes tenant from the token and publishes one BusinessCommand per order item):

```
POST /tmf-api/serviceOrdering/v5/serviceOrder
Authorization: Bearer <OAuth2 token: client=mvno-alpha-bss, tenant=mvno-alpha, scope=dxps:order:write>
Idempotency-Key: 6f1c5f5e-1c1a-4e8e-9a7e-2b3c4d5e6f70
{
  "externalId": "MVNO-ALPHA-2026-0001234",
  "priority": "1",
  "category": "mobile",
  "serviceOrderItem": [
    { "id": "1", "action": "add",
      "service": { "@type": "Service",
        "serviceSpecification": { "id": "CreateSubscriber5G", "version": "3.2.0" },
        "serviceCharacteristic": [
          { "name": "supi",   "value": "imsi-416770000000001" },
          { "name": "msisdn", "value": "962790000001" },
          { "name": "plan",   "value": "5G-100GB" } ] } },
    { "id": "2", "action": "add",
      "service": { "serviceSpecification": { "id": "AddVoNR" } },
      "serviceOrderItemRelationship": [ { "relationshipType": "dependsOn", "orderItem": { "itemId": "1" } } ] }
  ]
}
-> 201 Created  { "id": "0192f1c2-...", "state": "acknowledged", "@type": "ServiceOrder", ... }
   (Kafka: topic dxps.bc.p1, key "mvno-alpha|supi:imsi-416770000000001", header tenant=mvno-alpha)
```

Host-operator admin acting for an MVNO (requires scope dxps:tenant:any; audited in audit_event with acting_for_tenant = true):

```
POST /tmf-api/serviceOrdering/v5/serviceOrder
Authorization: Bearer <token: client=host-noc, scope=dxps:tenant:any>
X-Tenant: mvno-alpha
```

TMF688 notification to the tenant's registered hub listener:

```
{ "eventId": "...", "eventTime": "2026-10-05T10:15:30.120Z",
  "eventType": "ServiceOrderStateChangeEvent",
  "event": { "serviceOrder": { "id": "0192f1c2-...", "externalId": "MVNO-ALPHA-2026-0001234",
                               "state": "completed", "completionDate": "..." } } }
// delivered only to hub_subscription rows where tenant = 'mvno-alpha'
```

Example southbound 5G SBA call (UDR authentication subscription; payload abbreviated, key material by reference only):

```
PUT /nudr-dr/v2/subscription-data/imsi-416770000000001/authentication-data/authentication-subscription
:authority: udr01.5gc.mnc077.mcc416.3gppnetwork.org
authorization: Bearer <NRF access token>
3gpp-Sbi-Correlation-Info: ...      x-idempotency-key: tsk-...:1
{ "authenticationMethod": "5G_AKA", "encPermanentKey": "<HSM ref>", "protectionParameterId": "...",
  "algorithmId": "milenage", "encOpcKey": "<HSM ref>" }
```

6.13 Error model

Code	Meaning	TMF641 mapping	Legacy analogue
DXPS-1001	Unknown command spec / version	rejected	RC 100000 (no setup)
DXPS-1002	Parameter validation failed	rejected	RC 100001/100002 (syntax/undefined param)
DXPS-1003	No NE / endpoint resolvable	failed	RC 100003 (no queue)
DXPS-1004	Unknown, suspended or mismatched tenant (token / header / payload)	rejected (HTTP 403)	-
DXPS-1005	Tenant not entitled to the NE or operation (tenant_ne_access)	rejected	-
DXPS-1006	Tenant quota exceeded	rejected (HTTP 429, Retry-After)	-
DXPS-2001	Entity sequence gap timeout	held	-
DXPS-3xxx	NE business error (normalised from NE code)	failed / partial	Legacy response-message mapping
DXPS-4001	Retries exhausted	failed	Error 120 / transport errors
DXPS-5001	Compensation failed	failed + TMF642 alarm	ROLLBACK_FAILED 80 / 85

6.14 Performance and capacity model

Item	Baseline	Scale-out rule
Orchestrator pods	12 (3 AZ x 4)	+1 pod per ~800 BC/s; max = sum of tier partitions
Adapter pods (sba)	12	Bound by NE TPS, not CPU; ~3k tasks/s per pod with HTTP/2 multiplexing
Kafka brokers	6 (NVMe, 25 GbE)	Add brokers and reassign partitions (Cruise Control)
PostgreSQL	1 primary 32 vCPU + sync standby	~15k tx/s with batched plans; Citus beyond that
Payload size	BC ~1-2 KiB, task ~1-4 KiB	lz4 ~2-3x
Tenants	Up to ~200 tenants in the shared pool	Dedicated partitions/topics for tenants above ~10% of volume; dedicated DxPS cell for strict-isolation contracts

6.15 Testing strategy

- Unit and property tests for planner/sequencer (random DAGs, order invariants, coalescing).
- Contract tests from 3GPP/TMF/GSMA OpenAPI specs (generated clients + schema validation).
- Tenant isolation suite: for every API, query and Kafka consumer, run as tenant A against tenant B data (expect zero rows / 403); RLS bypass checks; header/payload tenant mismatch rejection.
- Noisy-neighbour test: tenant A runs a P3 bulk load at 10x quota while tenant B's P0/P1 SLOs are asserted.
- NE simulators: rewrite the legacy simulator idea (CMD -> RESP maps in IN_SIM, the HLR simulator, NPM async callbacks) as Go/WireMock stubs for SBA, NETCONF (netopeer2), gNMI, ES2+ with latency and fault injection.
- Integration: Testcontainers (Kafka KRaft, PostgreSQL) in CI.
- Chaos: broker kill, AZ loss, PostgreSQL failover, consumer rebalance during load. Assert no reorder for an entity and no duplicate NE effect.
- Performance: k6 / franz-go bench for 2x NFR load; track p99 per stage via exemplars.

6.16 Console API and web UI

The console (section 5.14) is two processes. The console API (cmd/console, internal/console) is a Go HTTPS server bound to loopback (127.0.0.1:8446) with a certificate issued from the platform CA. The web UI (dxps/console) is a Next.js 16 application on port 8092 whose server side acts as a backend-for-frontend: browser calls to /bff/* are forwarded to /api/v1/* on the console API.

```
browser --HTTPS, cookie dxps_console + x-dxps-csrf--> console UI (Next.js BFF, :8092)
        --HTTPS, Authorization: Bearer <session>-----> console API (Go, 127.0.0.1:8446)
              |-- PostgreSQL as dxps_ops (all tenants; tenant / NE / entitlement updates + audit_event)
              |-- Kafka admin + dxps.dlq consumer/producer (lag, dead letters, redrive)
              |-- stats ports 9201-9203, netsim admin 9199, Mosaic 8088
              |-- gateway 8443 (TMF641 orders, 2-minute tenant token, order read/write scopes)
              '-- ~/dxps-runtime/console/ (users.json, drafts/, audit.jsonl)
```

Concern	Design
Users	users.json (mode 0600); PBKDF2-SHA256 password hashes; at least 12 characters; five consecutive failures lock the account until an admin resets it. First user created with: console useradd -name NAME -roles admin
Sessions	Random bearer token per sign-in, held only in memory by the API. The BFF stores it in an HttpOnly, SameSite=Strict, Secure cookie and rejects state-changing requests without the x-dxps-csrf header
Authorisation	Ordered roles viewer < operator < configurator < admin; every route declares the minimum role and the guard checks it before the handler runs
Data access	Reads through the dxps_ops pool (cross-tenant). Each update runs in one transaction that reads the before row, applies the change and inserts audit_event (actor console:<user>, before and after)
Catalog drafts	A draft is a set of changed and deleted spec files over the embedded catalog. Validate overlays it, loads the full catalog and runs every spec example through the Lab; Export returns the files for internal/catalog/data
Template Lab	Runs the real planner (CEL conditions, DAG) and the adapter renderers for a tenant, spec and parameters; returns tasks, decisions and rendered NE requests; no NE is called and nothing is stored
Console audit	Sign-ins, failures and every change are appended to audit.jsonl in addition to the DxPS audit_event rows

Routes (/api/v1)	Minimum role
auth/login, auth/logout, auth/me, auth/password	none for login, otherwise viewer

GET overview, orders, orders/{tenant}/{id}, tasks, tenants, nes, catalog, catalog/spec, catalog/files, drafts, kafka, dlq, sims, loadgen, tests, hubs, audit, reports, settings	viewer
POST orders, orders/{tenant}/{id}/cancel, dlq/redrive, sims/fault, loadgen, probes	operator
PUT tenants/{id}, nes/{tenant}/{code}, nes/{tenant}/{code}/access; drafts create, save, delete, validate, export; POST lab	configurator
users (list, add, update, delete)	admin

7. Migration Strategy (legacy engine -> DxPS)

Phase	Activities	Exit criteria
0 Foundation	Kafka, PostgreSQL, CI/CD, observability, Schema Registry; catalog model; tenant service; onboard the host MNO as tenant 'host-mno'	DxPS SLOs met in staging
1 Catalog migration	Convert the legacy MML setup templates to GLOBAL catalog specs (tool parses command order, dependency sequence and command text); map MSISDN ranges to NE selection rules	100% of active command orders mapped and reviewed
2 Northbound switch	Host CRM publishes to Kafka (or TMF641). Temporary bridge: DxPS forwards commands for not-yet-migrated domains to the legacy interface table	PIL decommissioned for migrated channels
3 Domain by domain	5G SA/UDM-UDR first, then IMS, eSIM, BSS, transport/access. Shadow mode: dry-run plans compared with legacy MML output	Diff rate < 0.1%, then cut-over per domain
4 MVNO onboarding	Onboard MVNOs as tenants (OAuth2 clients, quotas, NE entitlements, catalog overrides, TMF688 hubs); migrate MVNO subscribers, tagging them with tenant and subscriber group	Each MVNO live with its own SLA report
5 Legacy NE retirement	NEs reachable only through MML/X.25/CORBA/SOAP-encoded are upgraded to modern NBI or decommissioned	No legacy SIM instance running
6 Decommission	Legacy Oracle schema archived (ora2pg for history if needed; history rows loaded with tenant 'host-mno'), Solaris/HP-UX hosts retired	Sign-off

8. Requirements Traceability

Requirement	Design element(s)
Business Command via Kafka	5.4 topics, 6.2 BusinessCommand contract, 6.6 ingestion
Command order	5.5, 6.5 key-lane executor, 6.7 entity sequencer
Priorities	5.6 tiers, 6.4 scheduler, per-NE reserved capacity
Transaction management	5.7, 6.8 saga, outbox/inbox, Kafka EOS
No PIL	5.3
Relations not one-to-one	5.8, 6.11 bc_task_link / task_dependency / bc_entity / ne_endpoint / tenant_ne_access
Go + PostgreSQL	6.1, 6.11
Latest Telco networks, no old protocols	5.10, 6.9
TMF standards	4.2, 6.12, 6.13
Scalable, very low latency	5.9, 5.11, 6.3, 6.14
Optimised Kafka features	6.3 (idempotence, transactions, KIP-848, static membership, compaction, tiered storage, share groups, client quotas)
Multi-tenant for MVNOs	5.2, 6.2 (tenant in every message), 6.4 tenant DRR, 6.9 per-tenant NE quotas, 6.11 RLS + partitioning
tenant varchar(40) in every table; PK (tenant, uuid)	6.11 DDL (dxps_tenant domain, composite PKs and FKs on all tables)
Target named DxPS; legacy ITS/Huawei terminology not used in the target	Whole DxPS design (chapters 4-9); legacy names appear only in the AS-IS chapter and migration sources

9. Risks and Open Decisions

#	Risk / decision	Mitigation / recommendation
R1	Some NEs expose only legacy interfaces	Inventory NE NBIs early (phase 0); vendor upgrade or replace; this is a programme risk, not a platform feature
R2	Source systems cannot supply entity_seq	Gateway assigns seq per entity at ingest (single partition owner), or arrival order is used

R3	Partition count growth remaps keys	Pre-size partitions; drain before expanding
R4	NE lacks idempotency	Read-before-write and compare; idempotency keys stored per task
R5	Kafka share groups maturity	Use only for bulk tier after GA
D1	Native Kafka producer vs TMF641-only northbound	Support both; recommend native for CRM/COM (lowest latency)
D2	Single PostgreSQL primary vs Citus	Start single primary + tenant partitioning; Citus (distributed by tenant) when > 15k tx/s
R6	One large MVNO dominates the shared pool	Per-tenant quotas and DRR; move it to dedicated partitions/topics or a dedicated cell
R7	Shared NEs cannot separate MVNO subscribers	Use subscriber groups / S-NSSAI / DNN tagging on the shared NE; otherwise the MVNO needs its own NE
D3	Shared pool vs dedicated cell per MVNO	Shared pool by default; dedicated cell only for contractual or regulatory isolation
D4	Same MSISDN in two tenants (porting between MVNOs)	Allowed: ordering is per (tenant, entity_key); port-out/port-in is two orders coordinated by the NPDB step

Appendix A - Glossary

Term	Definition
DxPS	Real-time Provisioning System - the target platform described in this document
Tenant	An operator brand served by DxPS (host MNO, full MVNO, light MVNO, enterprise); identified by tenant varchar(40)
MVNO	Mobile Virtual Network Operator - uses the host MNO's network (full MVNO: own core elements; light MVNO: host core)
GLOBAL	Reserved tenant for shared catalog defaults
RLS	PostgreSQL row-level security
DRR	Deficit round robin - fair scheduling across tenants
BC	Business Command - one requested change from CRM/COM or MVNO BSS (TMF641 order item)
NE task	One operation on one network element endpoint
entity_key	Canonical key whose commands must be ordered (subscriber, service, resource), scoped by tenant
PIL	Legacy Provisioning Interface Layer (CRM <-> legacy DB shuttle) - removed
MMLGEN / SIM	Legacy MML generator and Switch Interface Manager (HLRCOMM)
SBA	3GPP Service Based Architecture (HTTP/2 + JSON)
EOS	Kafka exactly-once semantics (idempotent producer + transactions)
Outbox / Inbox	Patterns for atomic DB change + event publish / consume dedupe
Saga	Long-running transaction made of local steps with compensations
ODA	TM Forum Open Digital Architecture

Appendix B - References

- TM Forum: TMF630, TMF632, TMF669, TMF633, TMF634, TMF638, TMF639, TMF640, TMF641, TMF642, TMF645, TMF652, TMF654, TMF666, TMF688, TMF701, TMF702, TMF921; ODA; SID; eTOM.
- 3GPP: TS 23.501, 23.502, 29.500, 29.501, 29.503, 29.504, 29.505, 29.507, 29.510, 29.512, 29.519, 29.522, 29.531, 29.536, 29.540, 29.562, 32.291, 33.501, 28.532, 28.541 (Rel-18/19).
- IETF: RFC 6241 (NETCONF), RFC 8040 (RESTCONF), RFC 7950 (YANG 1.1), RFC 8072 (YANG Patch), RFC 8299 (L3SM), RFC 8466 (L2SM), RFC 7396 / 6902 (JSON Merge Patch / Patch).
- OpenConfig gNMI; O-RAN WG10 O1; BBF TR-369 (USP), TR-385, TR-383, TR-459; GSMA SGP.22, SGP.32, Open Gateway / CAMARA.
- Apache Kafka 4.x: KIP-98 (EOS), KIP-345 (static membership), KIP-405 (tiered storage), KIP-848 (consumer group protocol), KIP-932 (share groups), KIP-500 (KRaft).
- Legacy ITS/Huawei sources (AS-IS study): src/prv/engine/MMLGEN (MMLGEN.h, CPRVInterface.pc, CTranslator.pc, CTranslationSetup.pc), src/prv/engine/SIM (hlrmain.c, hlrspc.*), src/prv/dispatchers/PIL, src/prv/adaptors/*, src/prv/simulators/*, engine/doc (MedProv 6.11.3 Rollback HLD/LLD).